# AWS VPC PEERING

## Hands-On Project Report

**Connecting private EC2 instances across two Amazon VPCs**

**Prepared by: SUMAN YADAV**

Project type: AWS networking lab

Services: Amazon VPC, Amazon EC2, VPC Peering

# Project Objectives

- Create two VPCs with non-overlapping IPv4 CIDR blocks.
- Separate resources into public and private subnets according to their roles.
- Use a bastion host as a controlled SSH entry point to a private instance.
- Establish VPC Peering between VPC-A and VPC-B.
- Configure routes in both VPCs and allow the required traffic through security controls.
- Validate private connectivity using ping and document the configuration with screenshots.

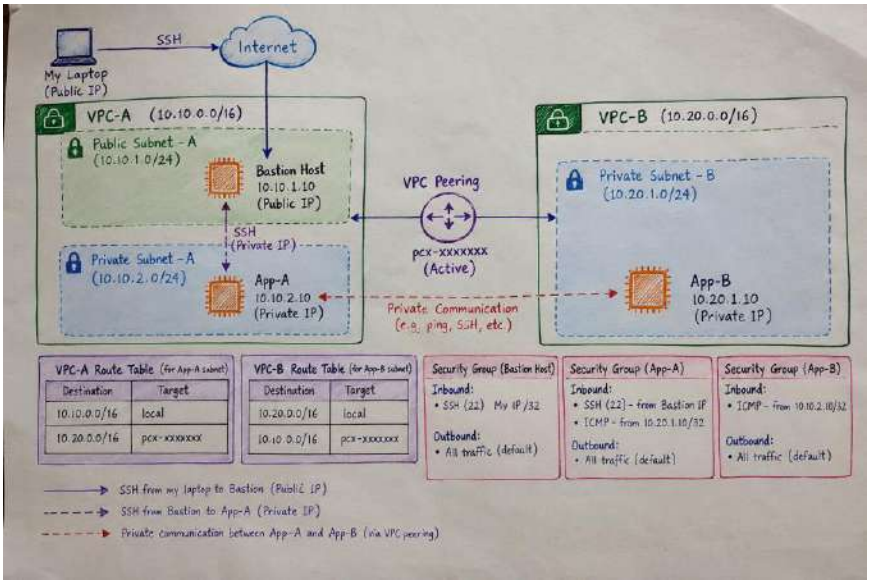
# Architecture and Network Design

The design contains two VPCs. VPC-A uses 10.10.0.0/16 and VPC-B uses 10.20.0.0/16. Since these ranges do not overlap, the VPCs can route traffic to each other through the peering connection. The bastion host is placed in a public subnet in VPC-A, App-A is in a private subnet in VPC-A, and App-B is in a private subnet in VPC-B.

| Component | Address / range | Purpose |
|---|---|---|
| VPC-A | 10.10.0.0/16 | First isolated network |
| VPC-B | 10.20.0.0/16 | Second isolated network |
| Public subnet A | 10.10.1.0/24 | Bastion host subnet |
| Private subnet A | 10.10.2.0/24 | App-A subnet |
| Private subnet B | 10.20.1.0/24 | App-B subnet |
| App-A | 10.10.2.10 | Source of ping test |
| App-B | 10.20.1.10 | Destination of ping test |

### How traffic flows

- 1. The administrator connects from the Windows laptop to the bastion host using SSH and a private key.
- 2. SSH is forwarded through the bastion host to App-A, which has a private IP address.
- 3. App-A sends ICMP Echo Requests to App-B's private IP address.
- 4. The route table in VPC-A sends traffic for 10.20.0.0/16 to the VPC peering connection.
- 5. VPC-B's route table provides the return route to 10.10.0.0/16. Security groups and network ACLs must also allow the relevant traffic.

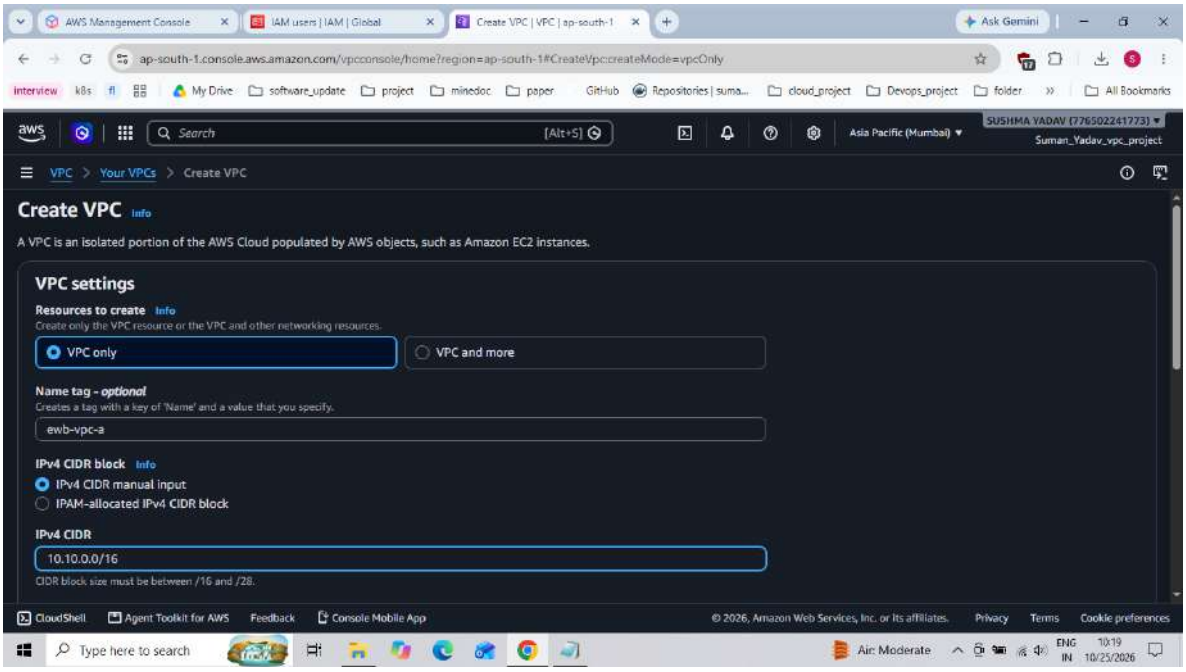


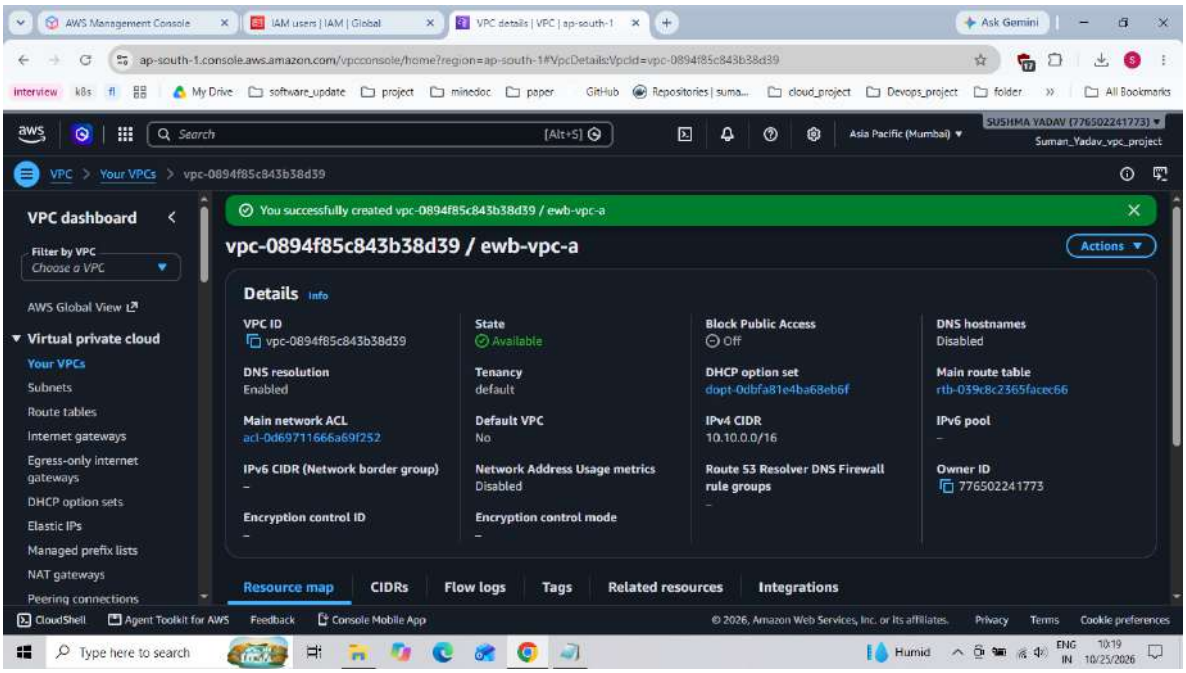

# Implementation Steps

## Creating the First VPC

I started by creating VPC-A.
I signed in to the AWS Management Console and opened the Amazon VPC service.
I navigated to **Your VPCs → Create VPC**.
I selected the option to create a VPC and entered the following details:

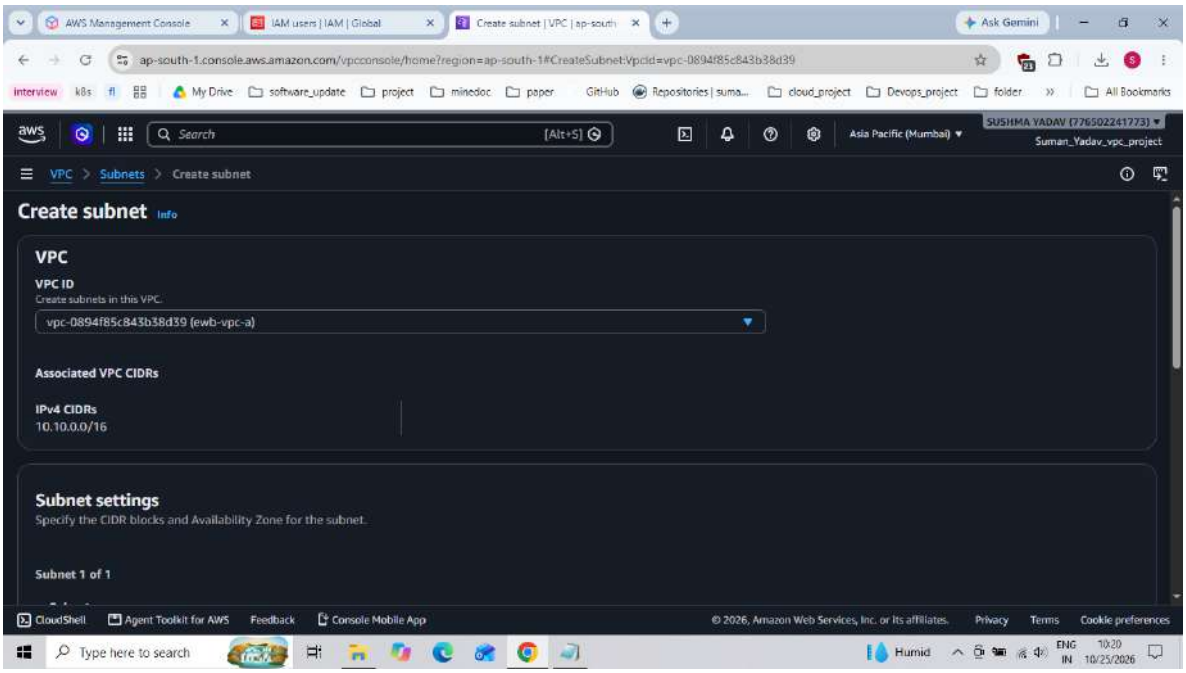
- Name tag: VPC-A
- IPv4 CIDR block: 10.10.0.0/16




AWS Management Console
IAM users | IAM | Global
VPC details | VPC | ap-south-1
Ask Gemini
ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#VpcDetails:VpcId=vpc-0894f85c843b38d39
Search
[Alt+S]
Asia Pacific (Mumbai)
SUSHMA YADAV (776502241773)
Suman_Yadav_vpc_project
VPC > Your VPCs > vpc-0894f85c843b38d39
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
You successfully created vpc-0894f85c843b38d39 / ewb-vpc-a
vpc-0894f85c843b38d39 / ewb-vpc-a
Actions
Details Info
VPC ID
vpc-0894f85c843b38d39
State
Available
Block Public Access
Off
DNS hostnames
Disabled
DNS resolution
Enabled
Tenancy
default
DHCP option set
dopt-0dbfa81e4ba68eb6f
Main route table
rtb-039c8c2365facec66
Main network ACL
acl-0d69711666a69f252
Default VPC
No
IPv4 CIDR
10.10.0.0/16
IPv6 pool
IPv6 CIDR (Network border group)
Network Address Usage metrics
Disabled
Route 53 Resolver DNS Firewall rule groups
Owner ID
776502241773
Encryption control ID
Encryption control mode
Resource map
CIDRs
Flow logs
Tags
Related resources
Integrations
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Type here to search
Humid
ENG
IN
10:19
10/25/2026

# Creating the Subnets

**After creating the VPCs, I created the subnets needed for the architecture.**

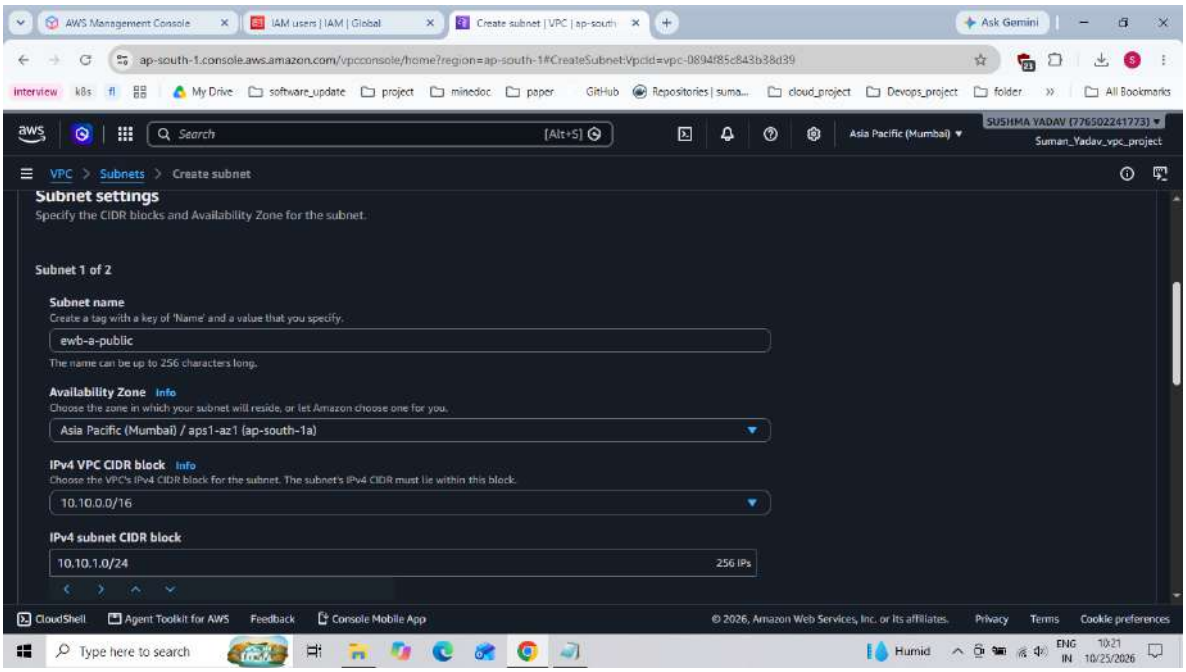



# Create the public subnet in VPC-A

I opened **Subnets → Create subnet**, selected VPC-A, and entered:

Subnet name: Public-Subnet-A

IPv4 CIDR block: 10.10.1.0/24



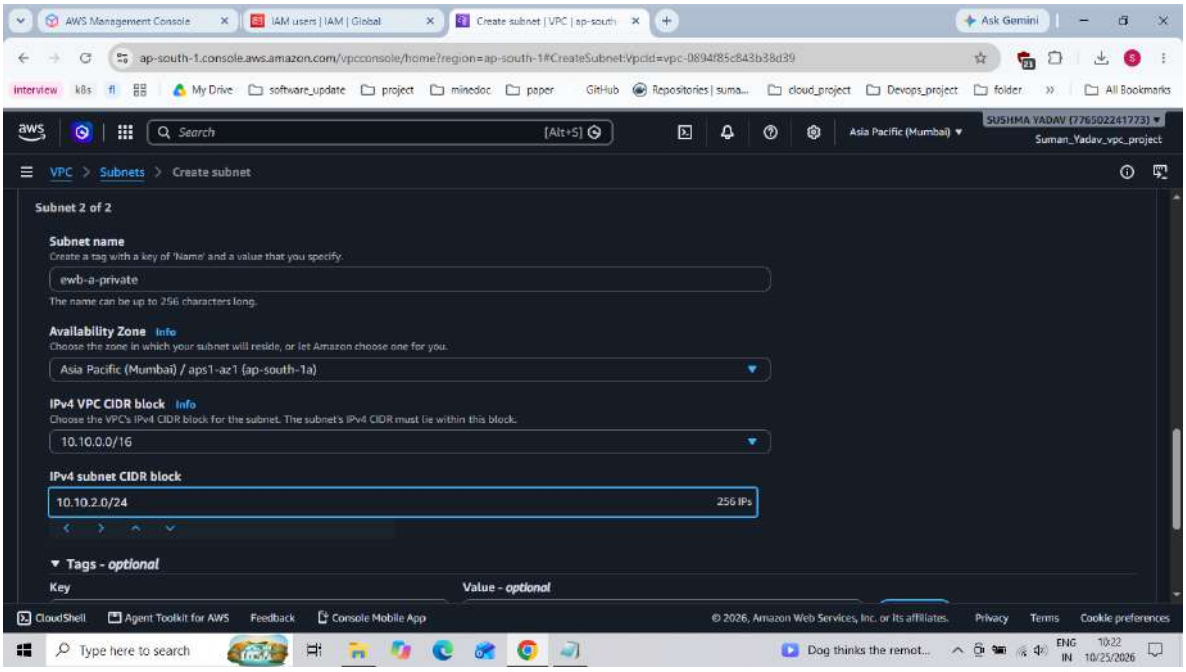
# Create the private subnet in VPC-A

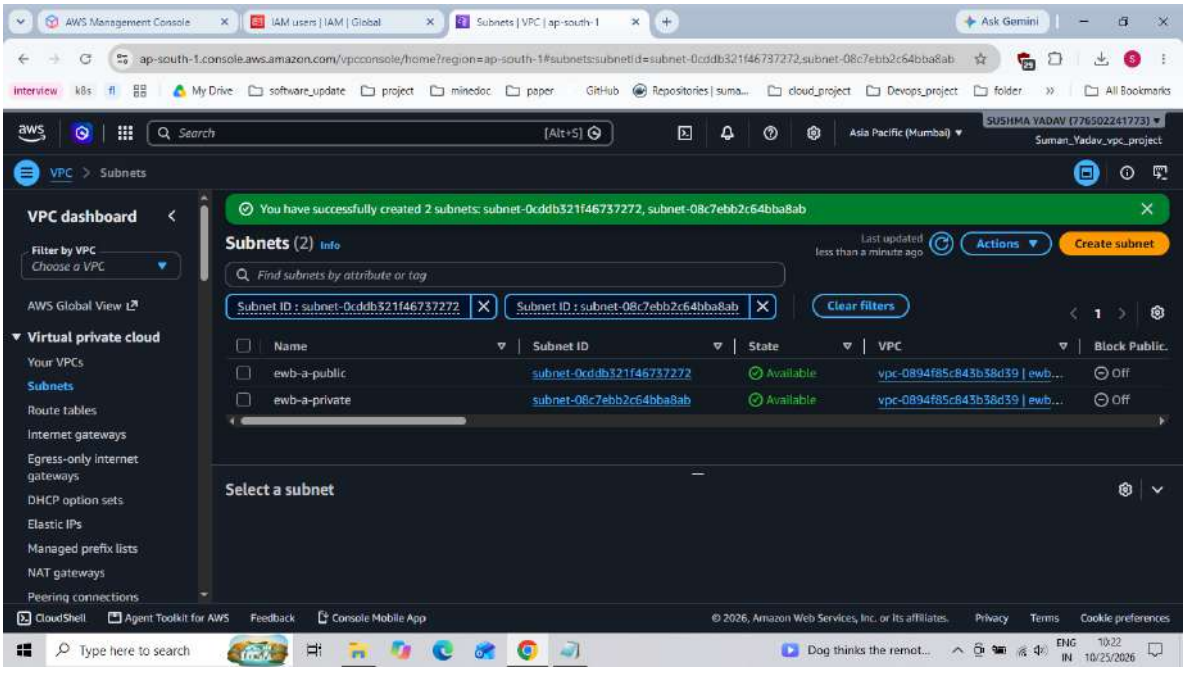
**I created another subnet in VPC-A:**

**Subnet name: Private-Subnet-A**

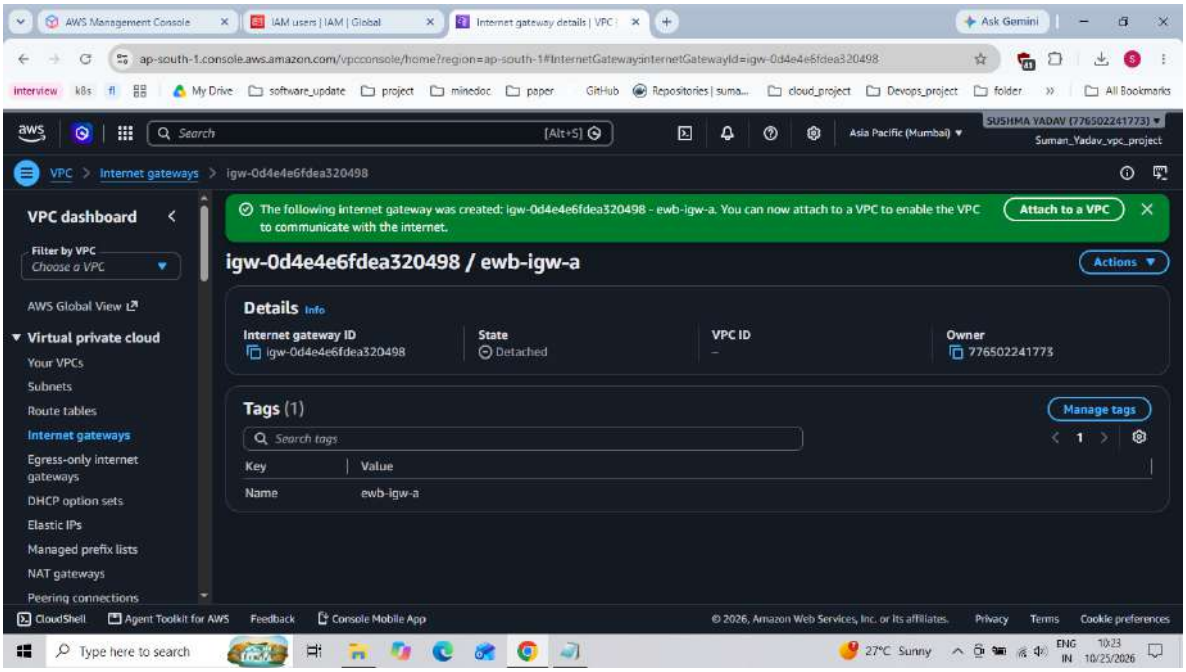
**IPv4 CIDR block: 10.10.2.0/24**

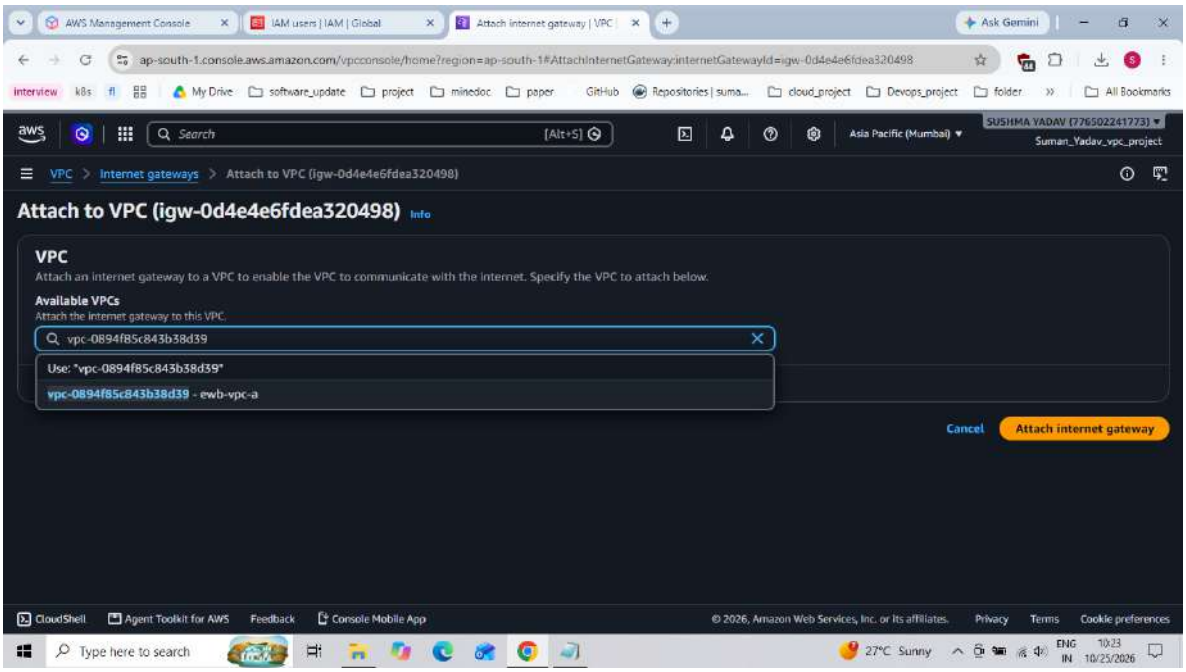
**This subnet was intended for App-A.**



# Create and attach the Internet Gateway



# Configure the public route

I opened the route table associated with the public subnet and checked that it contained:

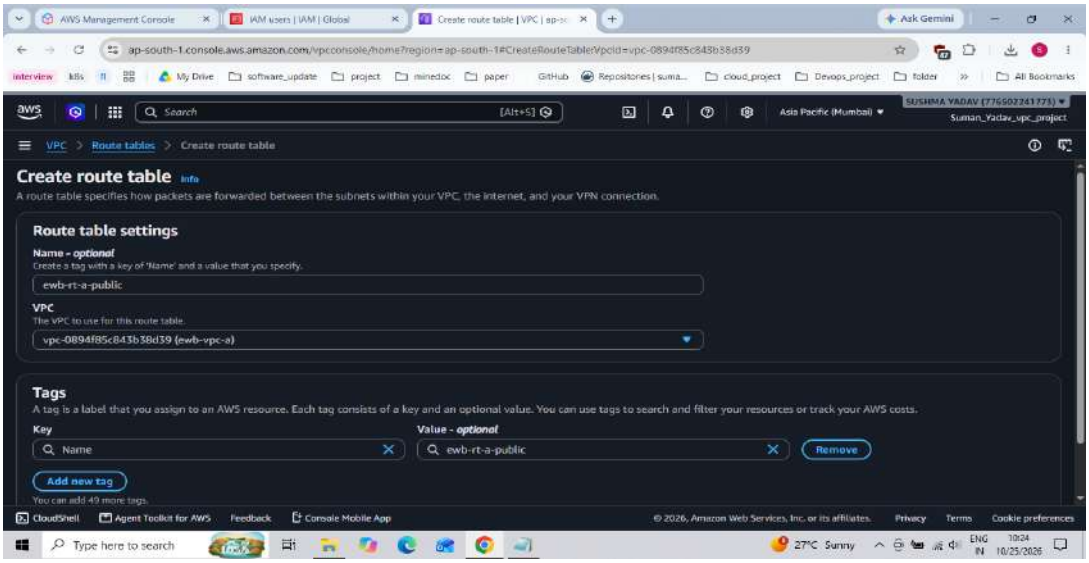
Name: ewb-rt-a-public

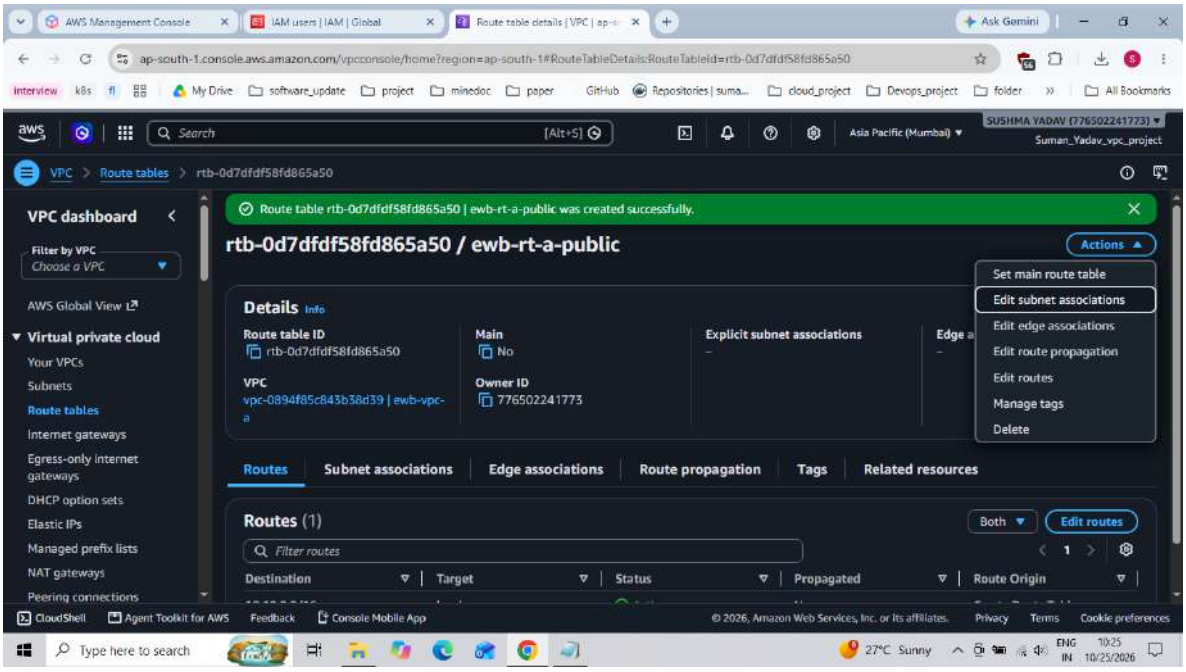
VPC: ewb-vpc-a

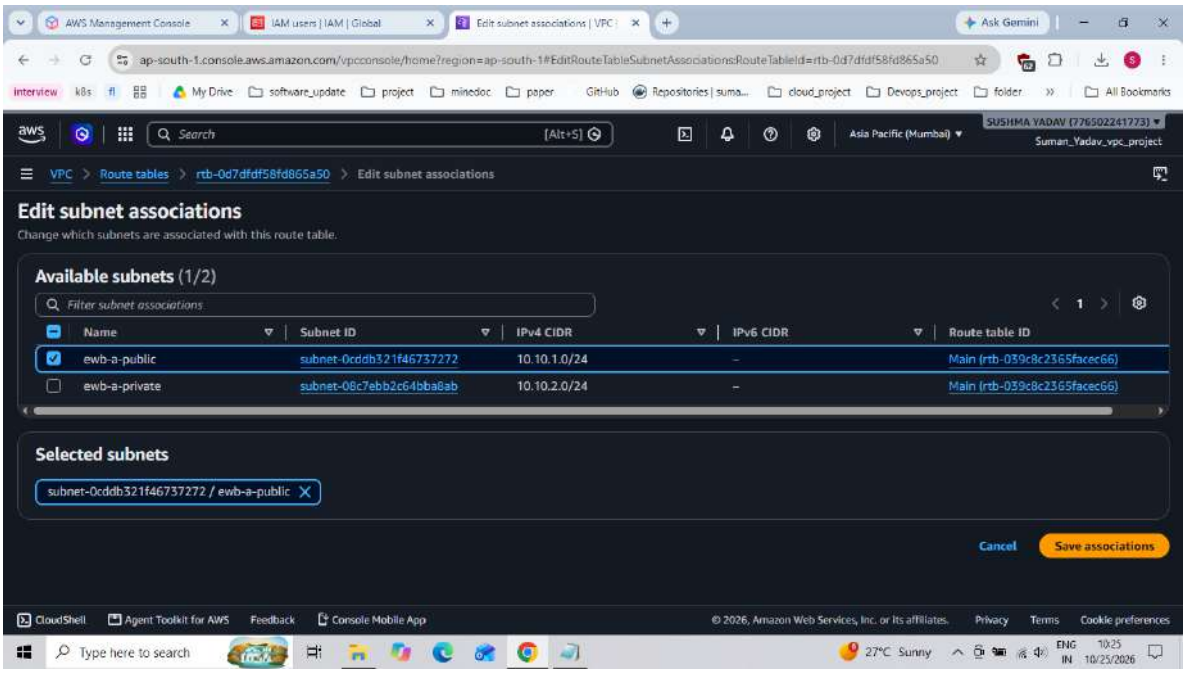
**Destination ➔ Target**

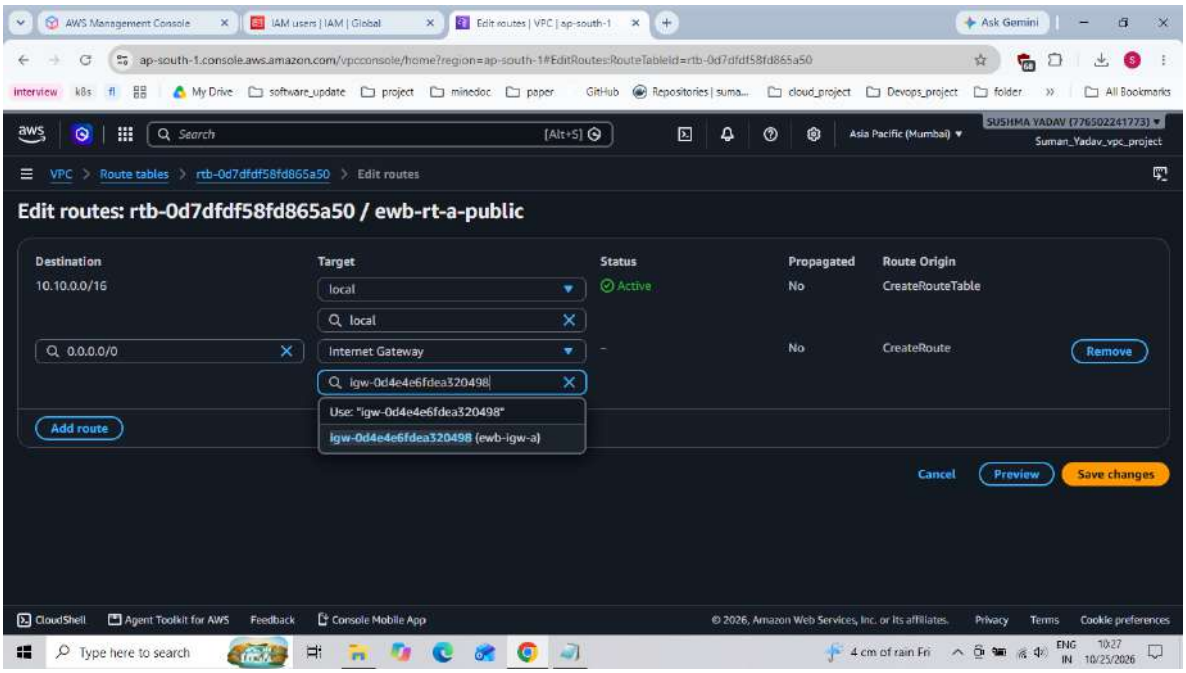
10.10.0.0/16 → Local

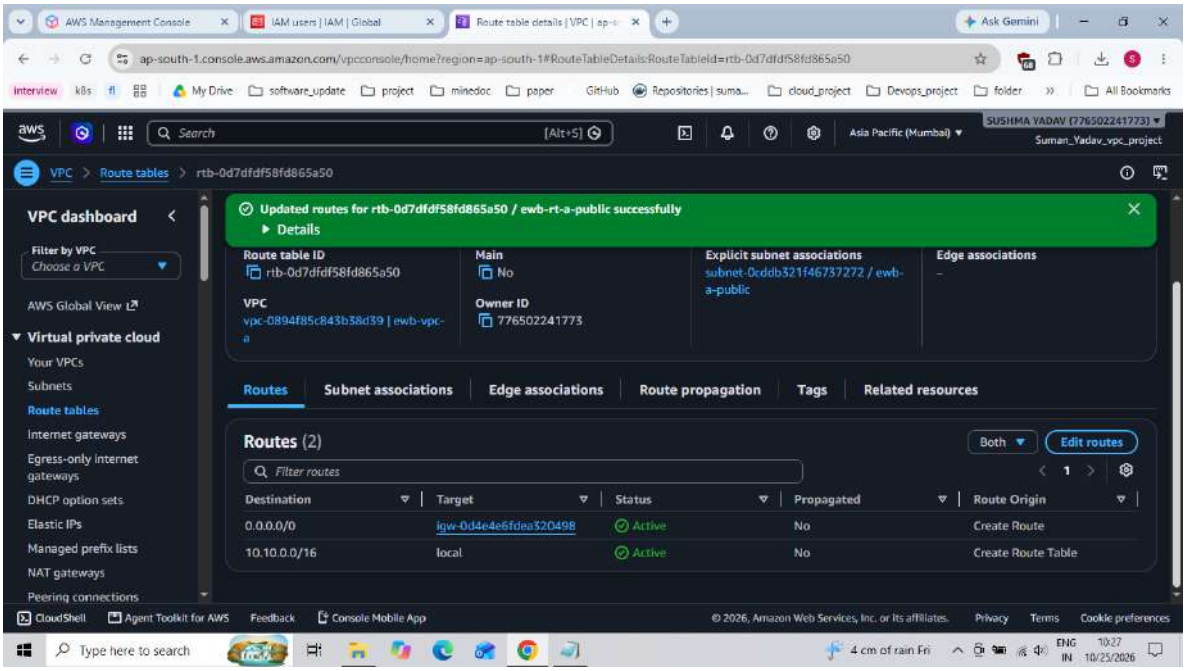
0.0.0.0/0 → Internet Gateway



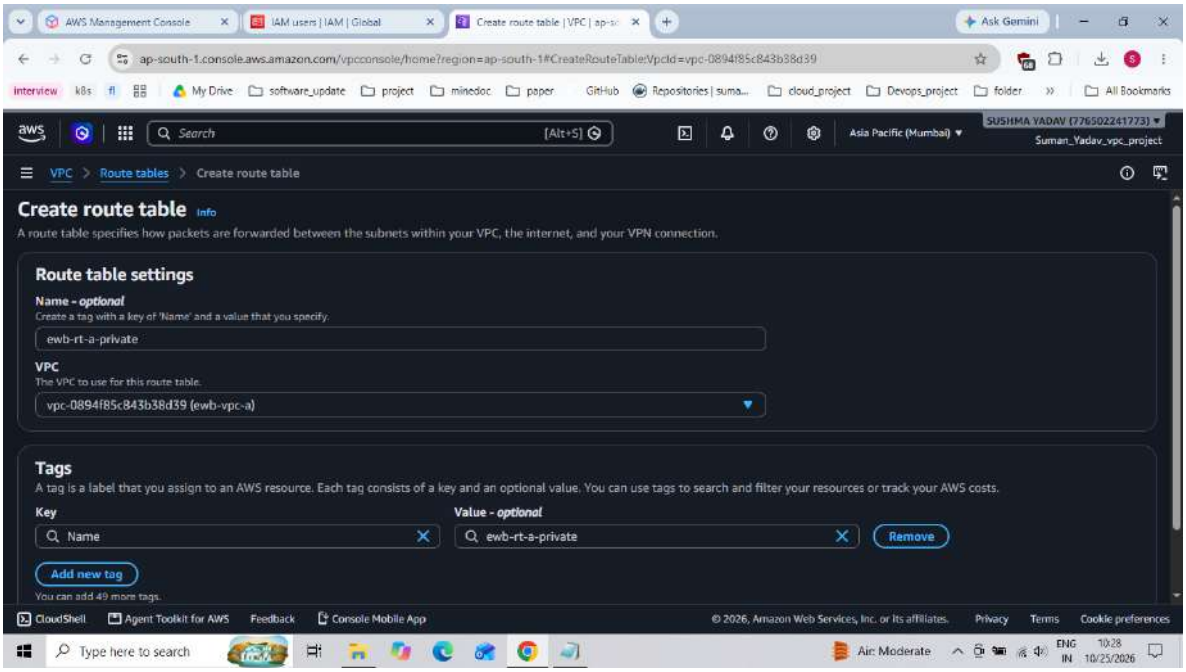



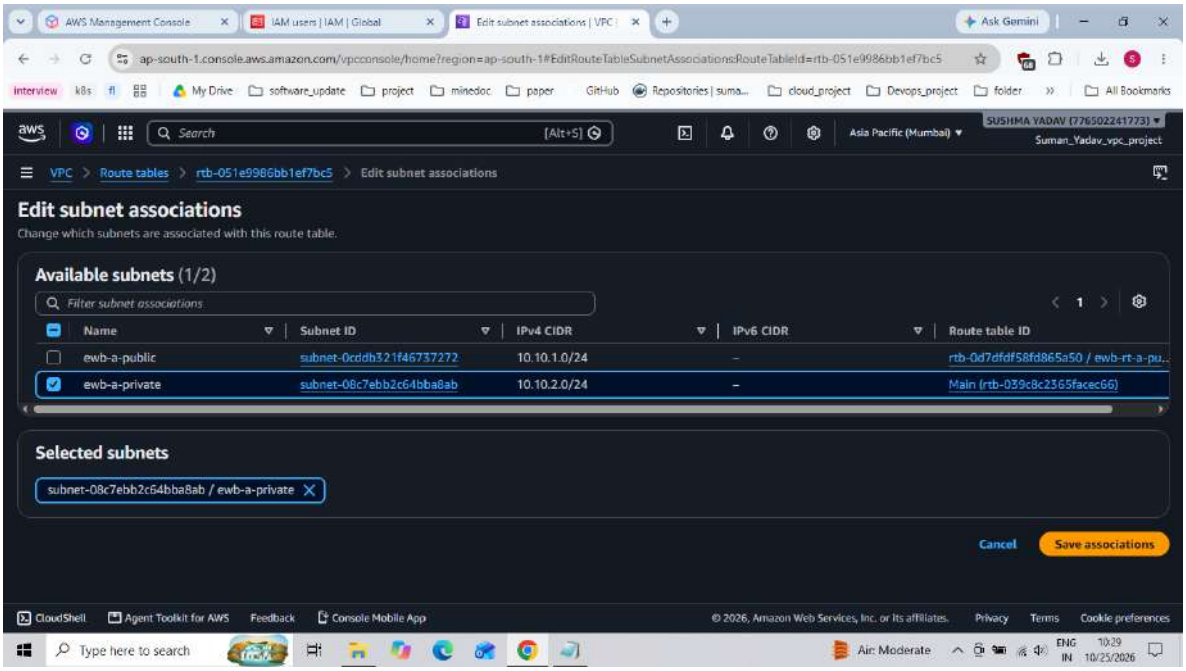

AWS Management Console
IAM users | IAM | Global
Edit subnet associations | VPC
Ask Gemini
ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#EditRouteTableSubnetAssociations:RouteTableId=rtb-0d7dfdf58fd865a50
Search
[Alt+S]
Asia Pacific (Mumbai)
SUSHMA YADAV (776502241273)
Suman_Yadav_vpc_project
VPC > Route tables > rtb-0d7dfdf58fd865a50 > Edit subnet associations
Edit subnet associations
Change which subnets are associated with this route table.
Available subnets (1/2)
Filter subnet associations
Name
Subnet ID
IPv4 CIDR
IPv6 CIDR
Route table ID
ewb-a-public
subnet-0cddb321f46737272
10.10.1.0/24
Main (rtb-039c8c2365facec66)
ewb-a-private
subnet-08c7ebb2c64bba8ab
10.10.2.0/24
Main (rtb-039c8c2365facec66)
Selected subnets
subnet-0cddb321f46737272 / ewb-a-public
Cancel
Save associations
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Type here to search
27°C Sunny
ENG
IN
10:25
10/25/2026


Edit routes: rtb-0d7dfdf58fd865a50 / ewb-rt-a-public
Destination
Target
Status
Propagated
Route Origin
10.10.0.0/16
local
Active
No
CreateRouteTable
0.0.0.0/0
Internet Gateway
No
CreateRoute
Remove
igw-0d4e4e6fdea320498
Use: "igw-0d4e4e6fdea320498"
igw-0d4e4e6fdea320498 (ewb-igw-a)
Add route
Cancel
Preview
Save changes


Updated routes for rtb-0d7dfdf58fd865a50 / ewb-rt-a-public successfully
Details
Route table ID
rtb-0d7dfdf58fd865a50
Main
No
Explicit subnet associations
subnet-0cddb521f46737272 / ewb-a-public
Edge associations
VPC
vpc-0894f85c843b38d39 | ewb-vpc-a
Owner ID
776502241773
Routes
Subnet associations
Edge associations
Route propagation
Tags
Related resources
Routes (2)
Both
Edit routes
Destination
Target
Status
Propagated
Route Origin
0.0.0.0/0
igw-0d4e4e6fdea320498
Active
No
Create Route
10.10.0.0/16
local
Active
No
Create Route Table

# Create the private route table



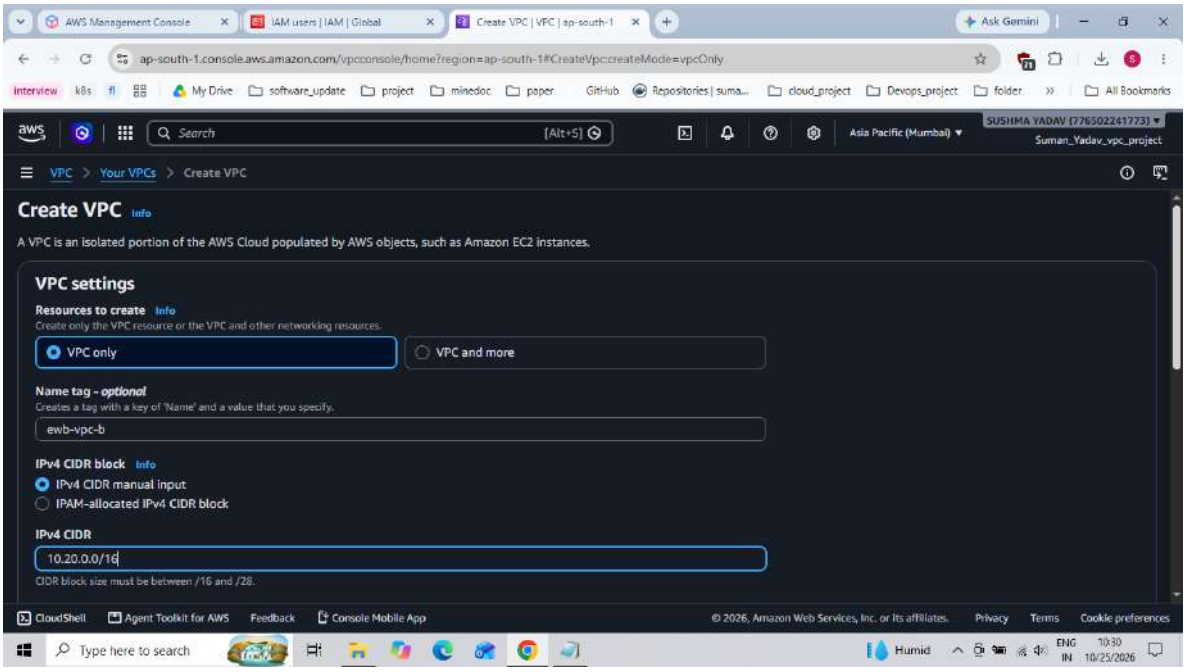



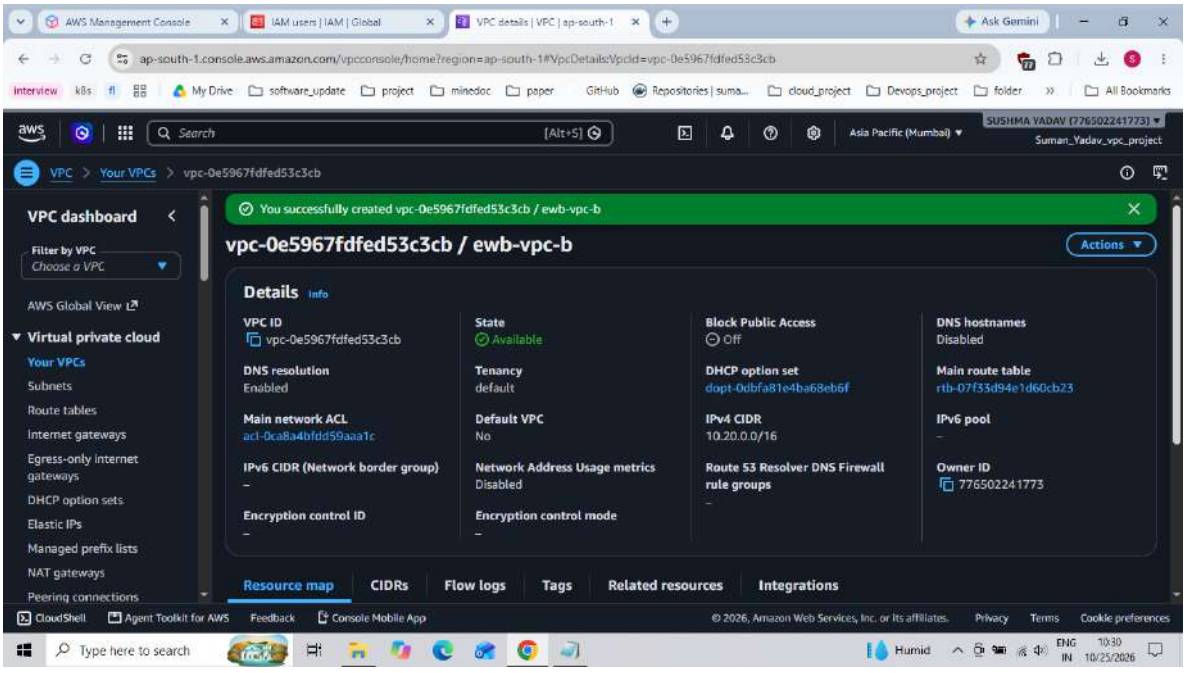
# Creating the Second VPC

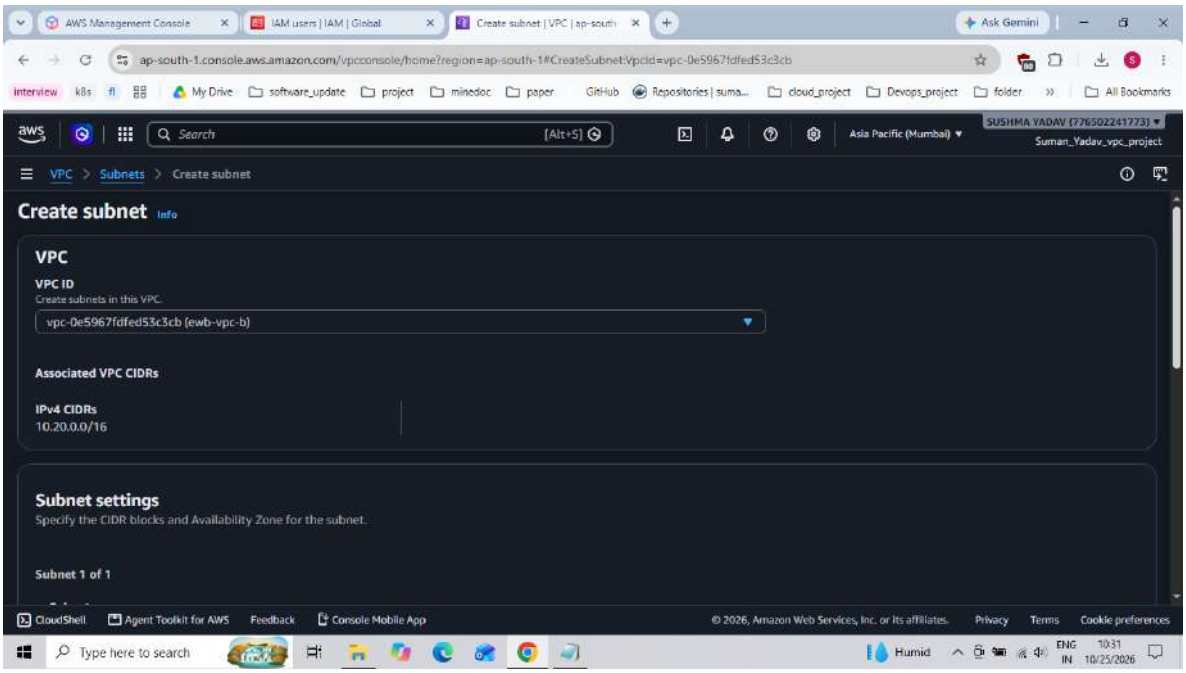
**Next, I created VPC-B.**

**I followed the same process:**

1. **Selected Your VPCs → Create VPC.**
2. **Set the IPv4 CIDR block to 10.20.0.0/16.**

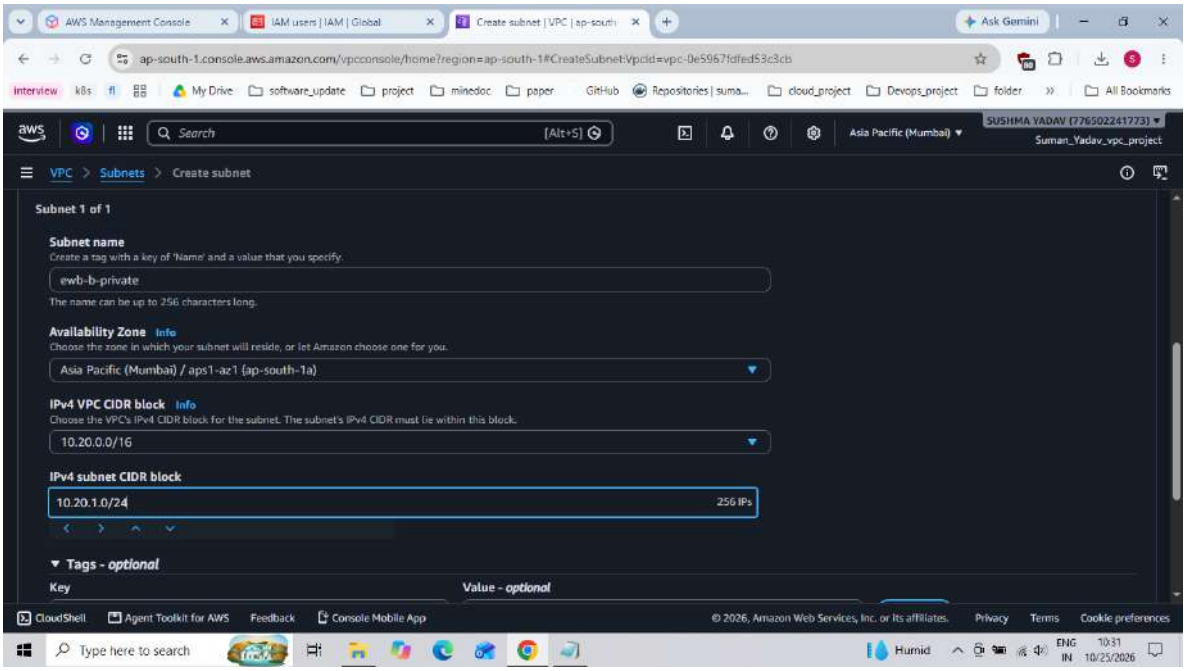


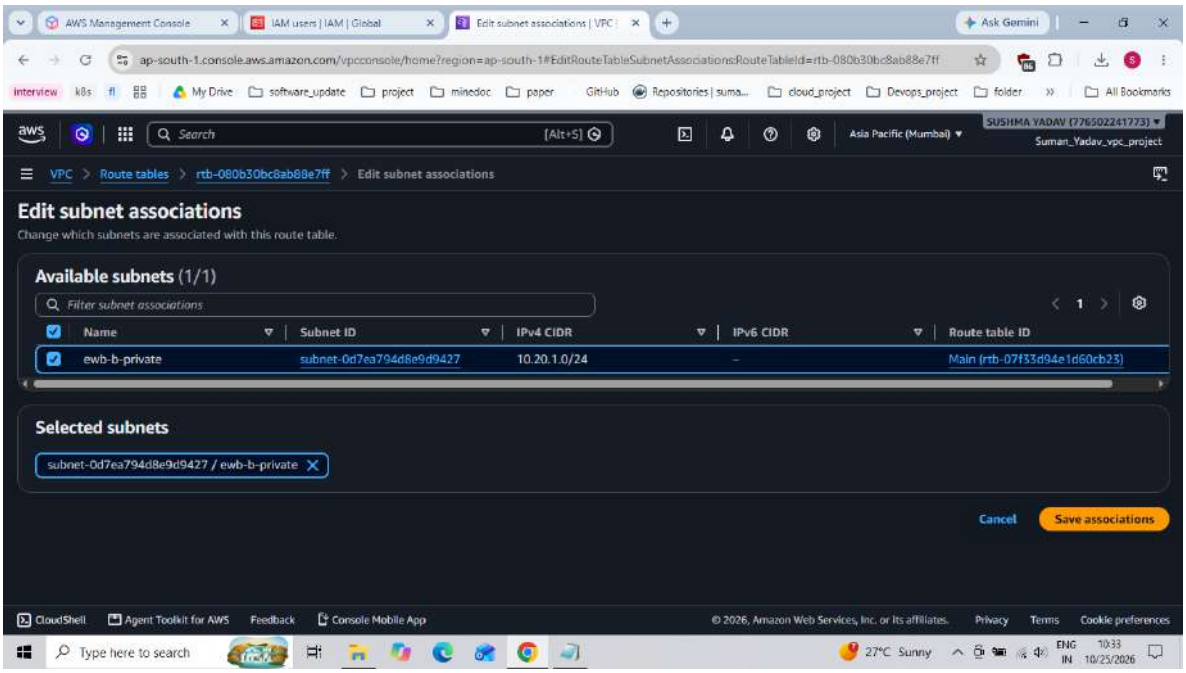



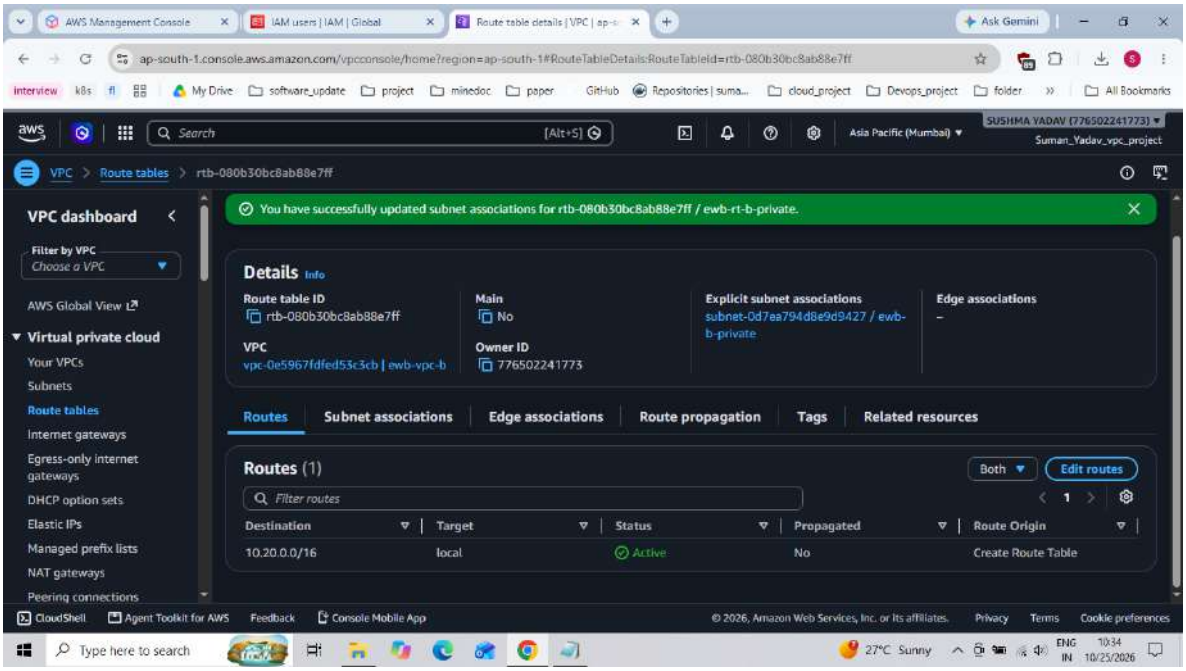



# Create the private subnet in VPC-B

**I selected VPC-B and created:**
Subnet name: Private-Subnet-B
IPv4 CIDR block: 10.20.1.0/24
This subnet was used for App-B.

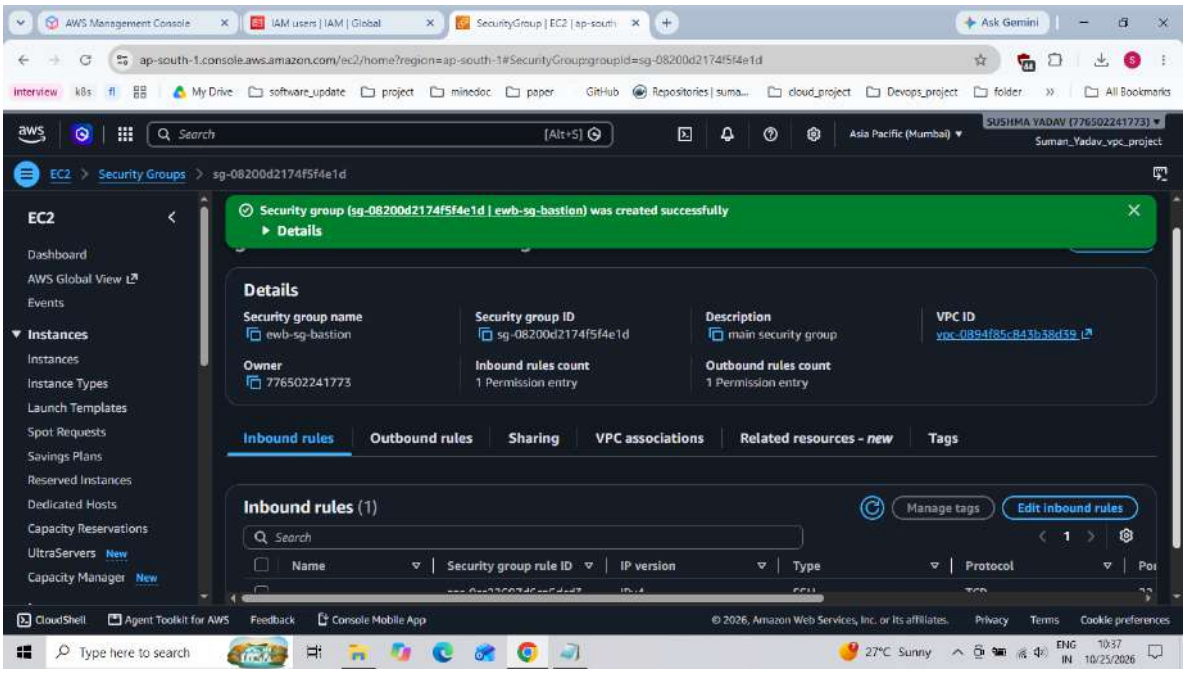




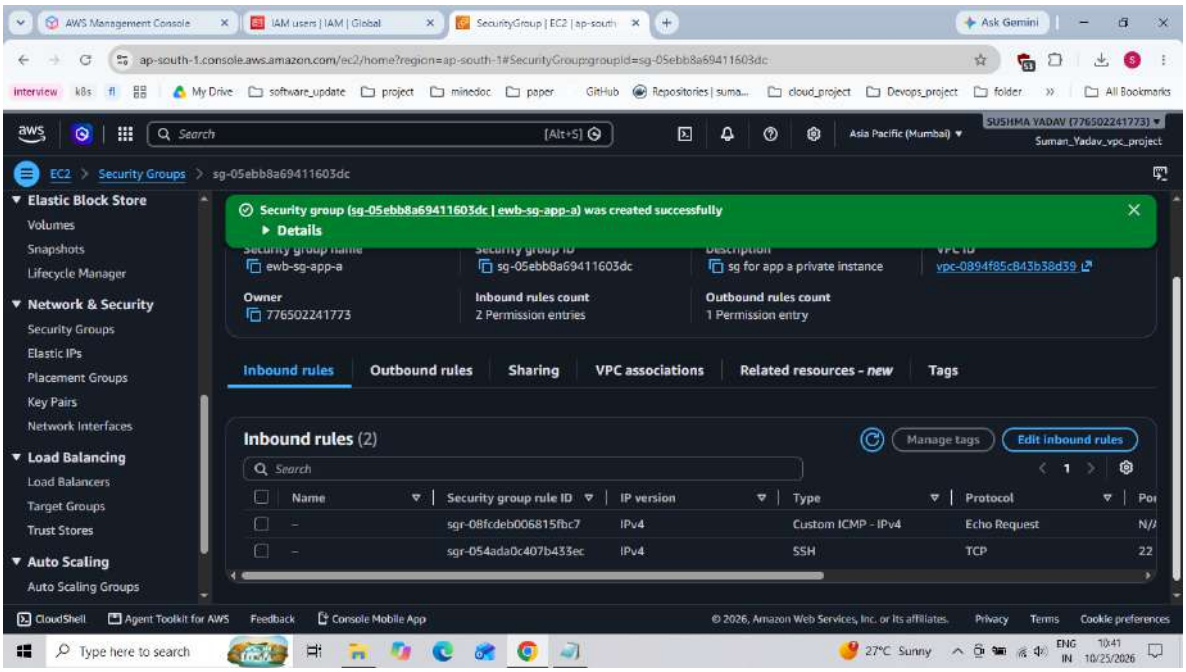




# Create the three security groups

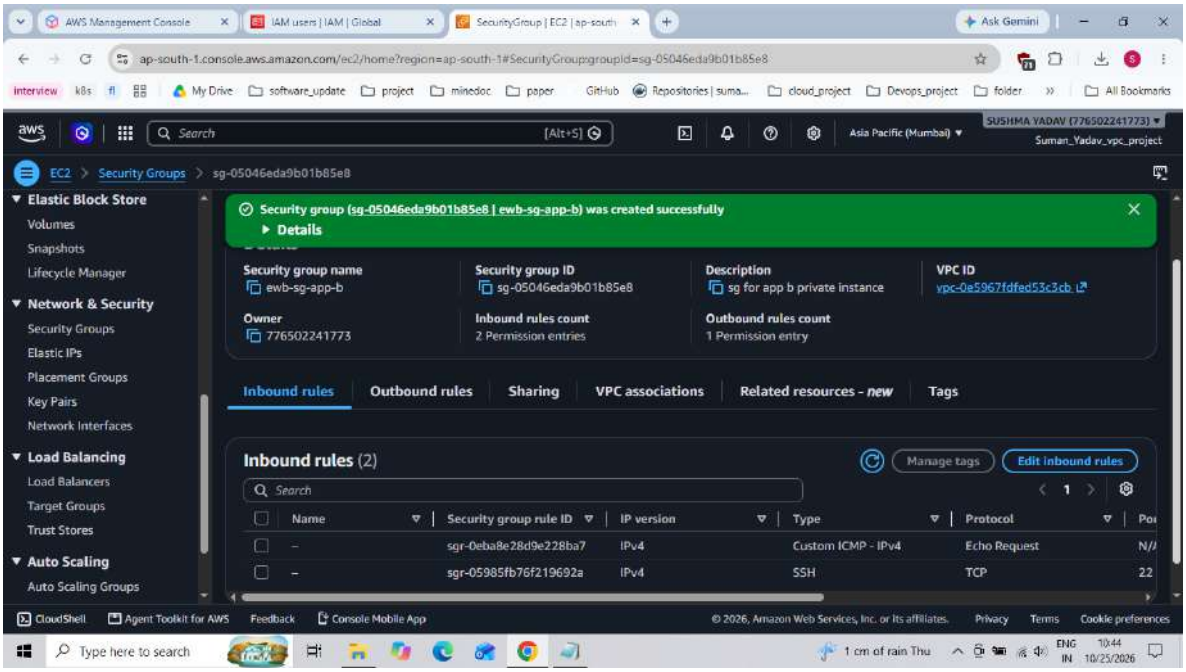
1. **Bastion host security group**




## 2. App-A security group



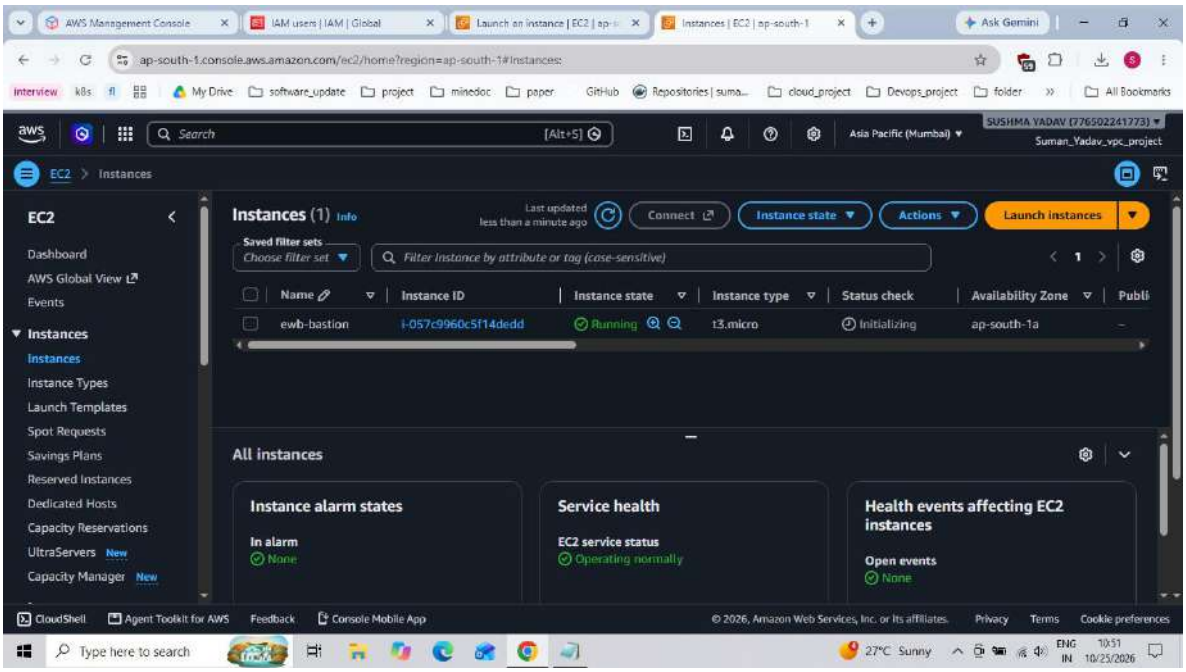
## 3. App-B security group



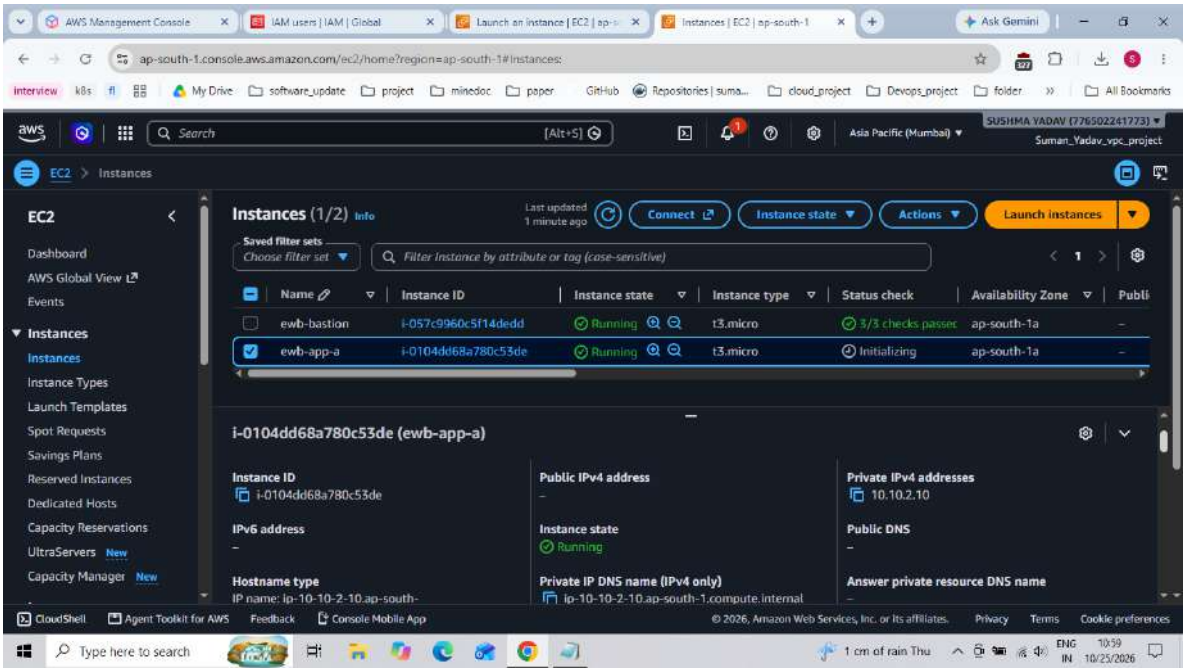

# Launch the three EC2 instances

I launched an EC2 instance in the public subnet of VPC-A.

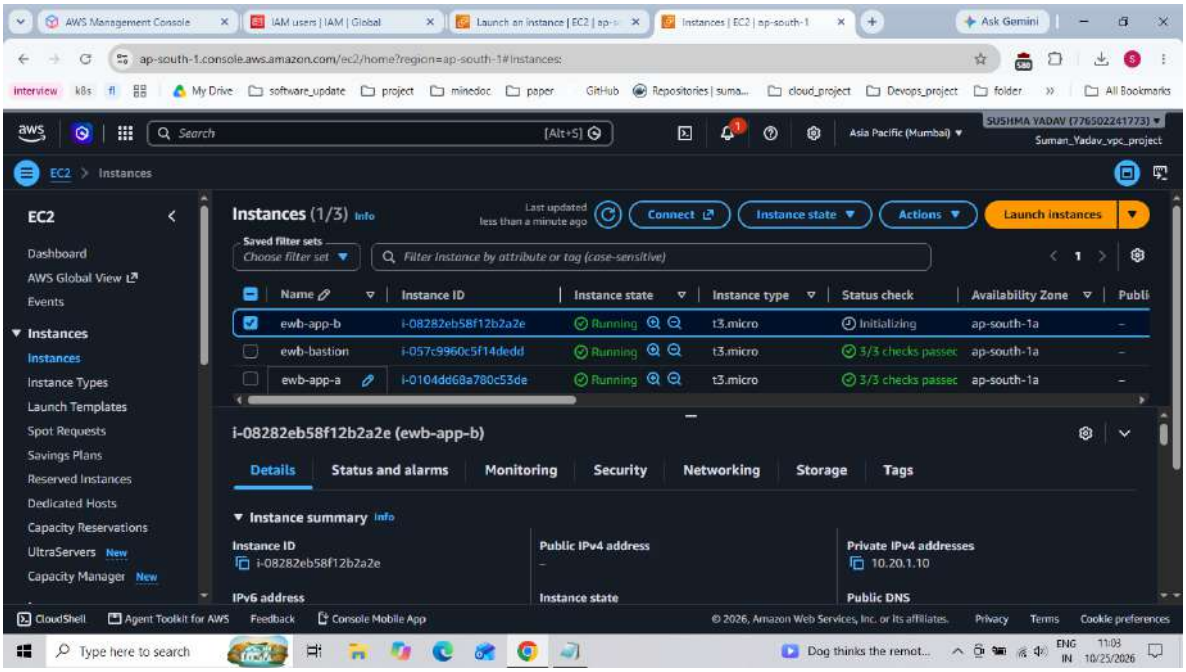
**Launch the bastion host**



## Launch App-A




## Launch App-B



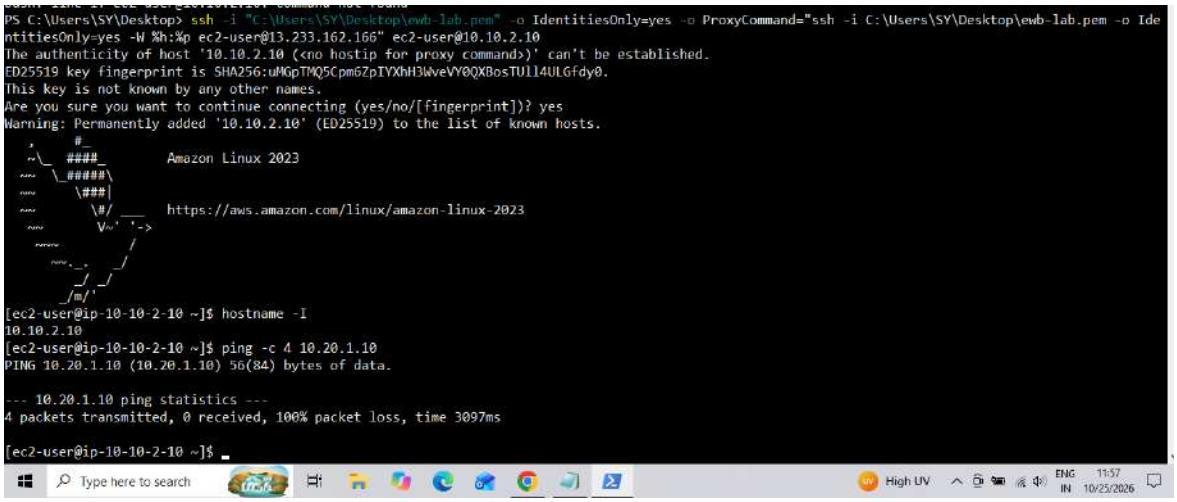
# Connect to the bastion host from Windows

**I used Windows PowerShell on my laptop to connect to the bastion host.**

The command structure was:

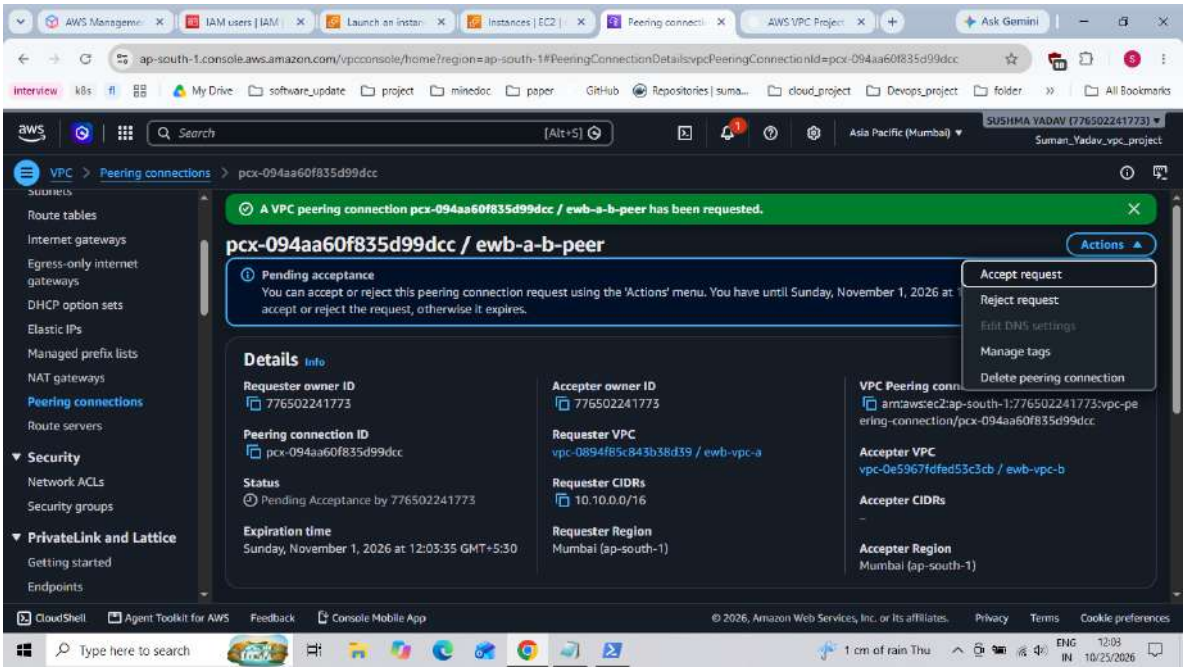
ssh -i "C:\Users\SY\Desktop\ewb-lab.pem" ec2-user@<BASTION_PUBLIC_IP>

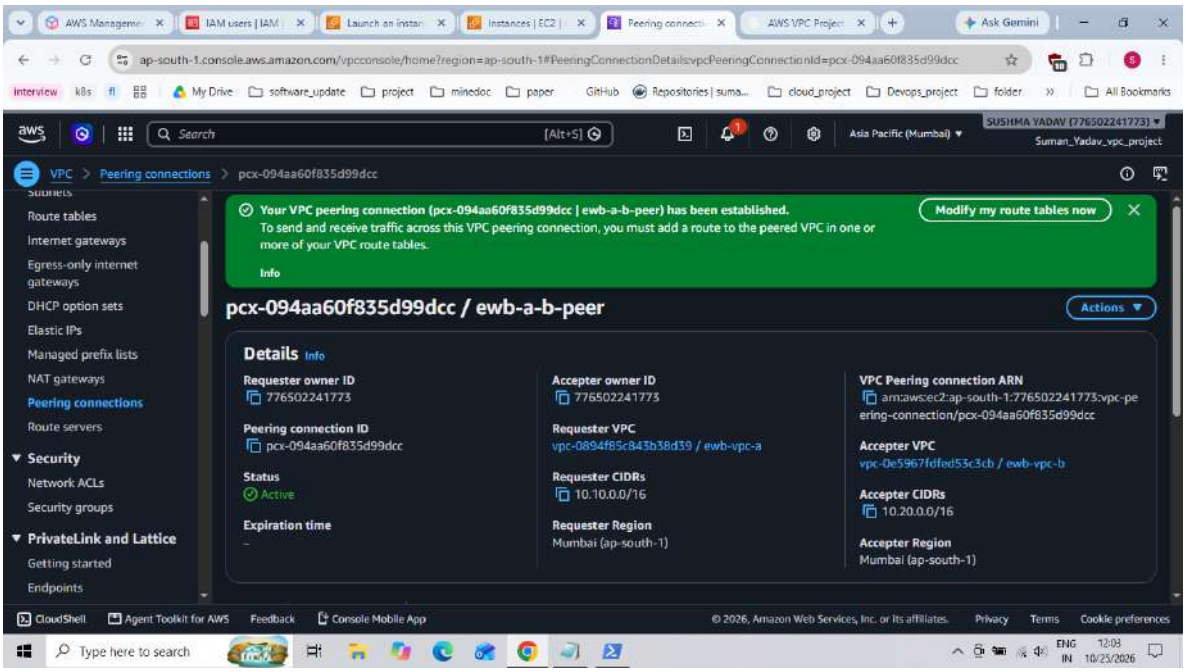



RIGHT NOW NO VPC PEERING SO, NO CONNECTIVITY

# Creating the VPC peering connection

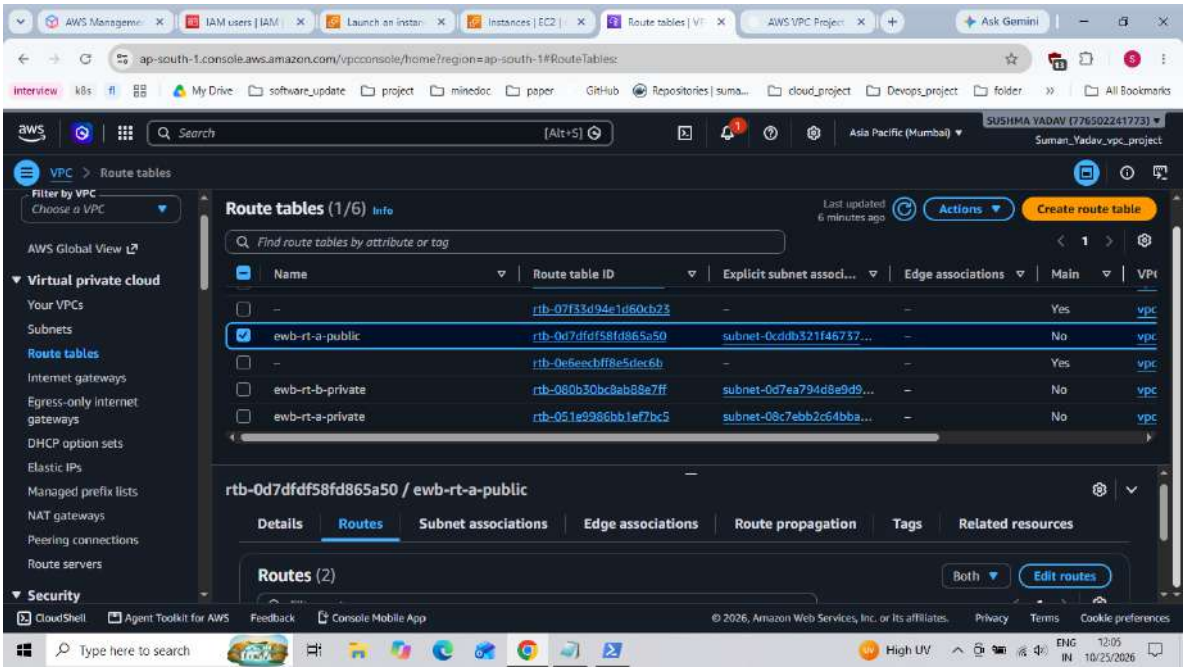


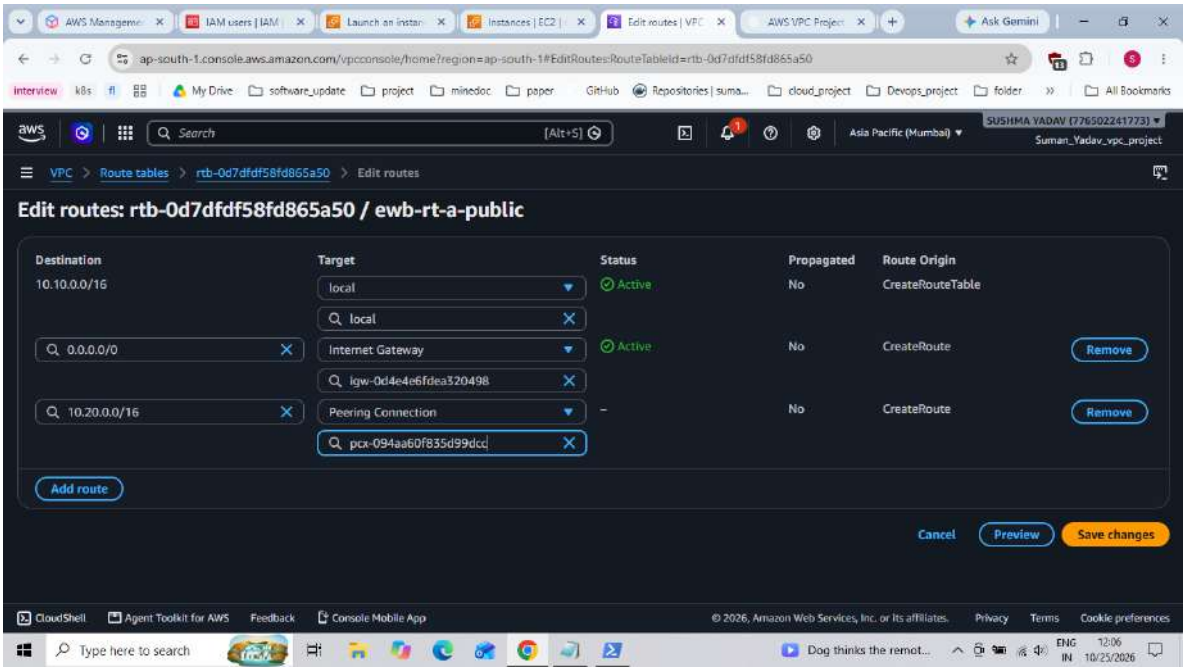



# Configuring the Route Tables for Peering

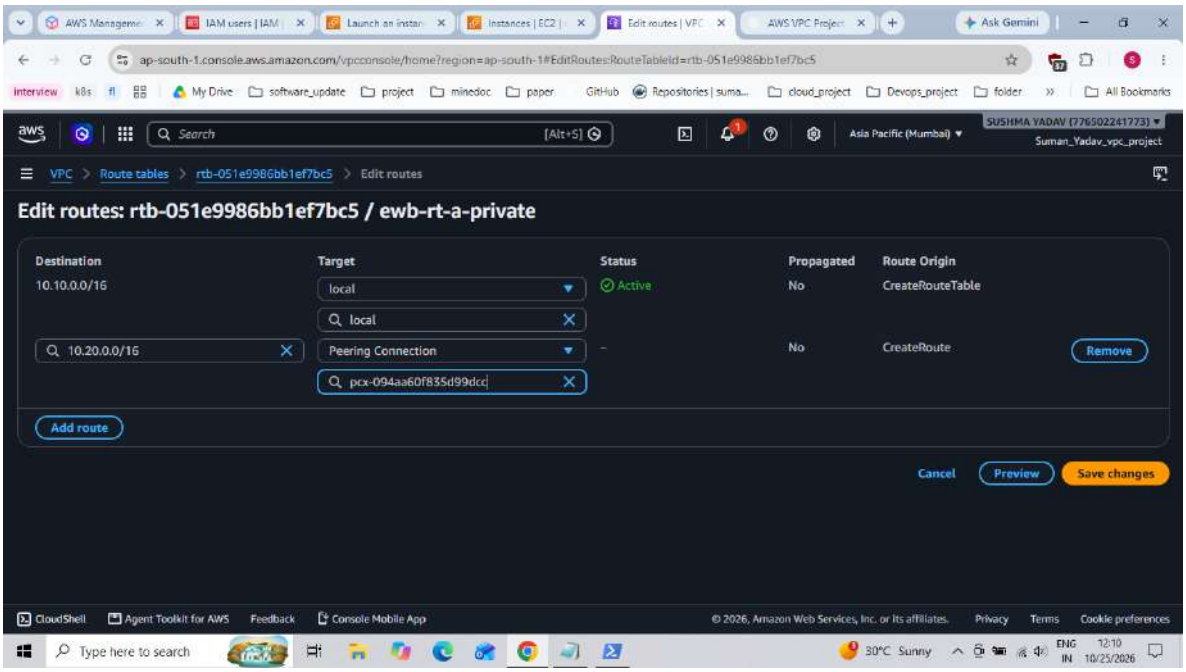
## Add routes to all three route tables

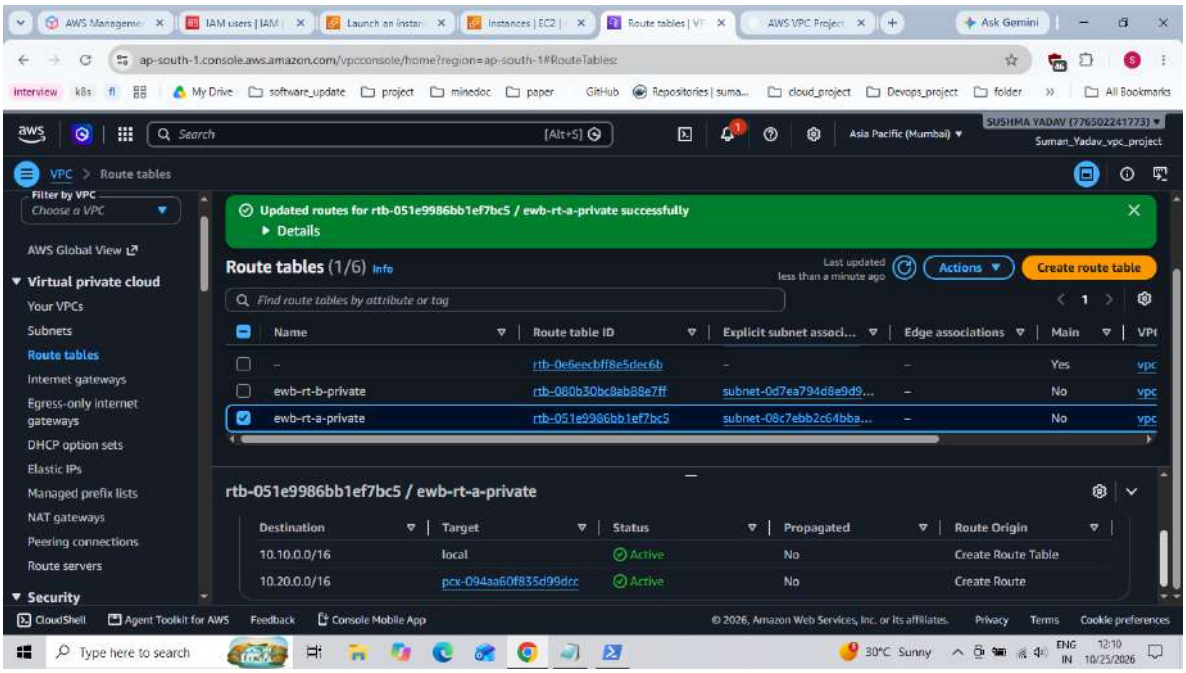
### Update ewb-rt-a-public

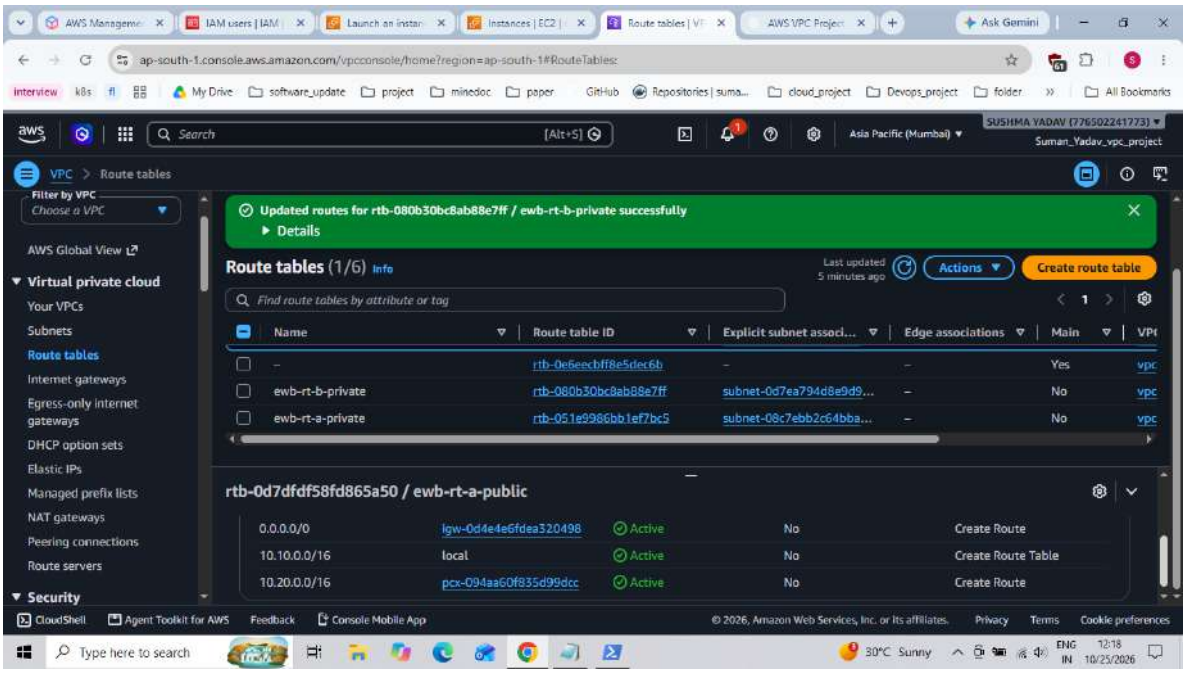


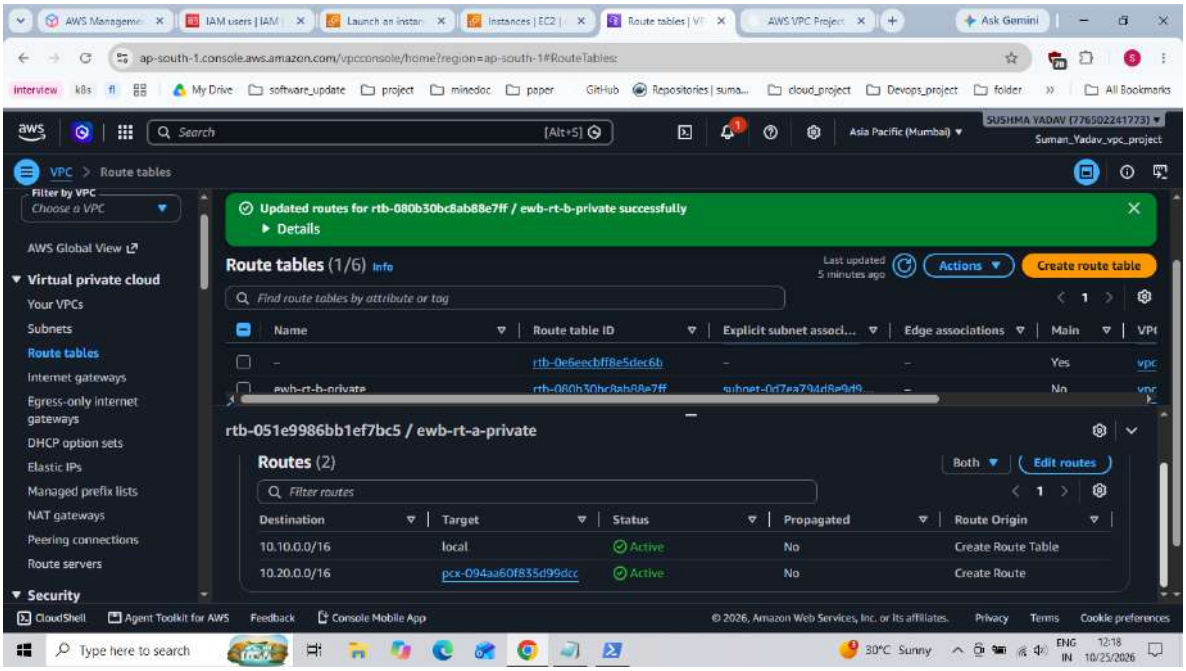
# Update ewb-rt-a-private






## Update ewb-rt-b-private




# Test private connectivity

### Connecting to App-A Through the Bastion Host

I used SSH ProxyCommand to route the SSH connection through the bastion host.

The command I used in Windows PowerShell was:

ssh -i "C:\Users\SY\Desktop\ewb-lab.pem" -o IdentitiesOnly=yes -o ProxyCommand="ssh -i C:\Users\SY\Desktop\ewb-lab.pem -o IdentitiesOnly=yes -W %h:%p ec2-user@<BASTION_PUBLIC_IP>" ec2-user@10.10.2.10

## Verify that I reached the correct instance

Once connected to App-A, I ran:

hostname -I

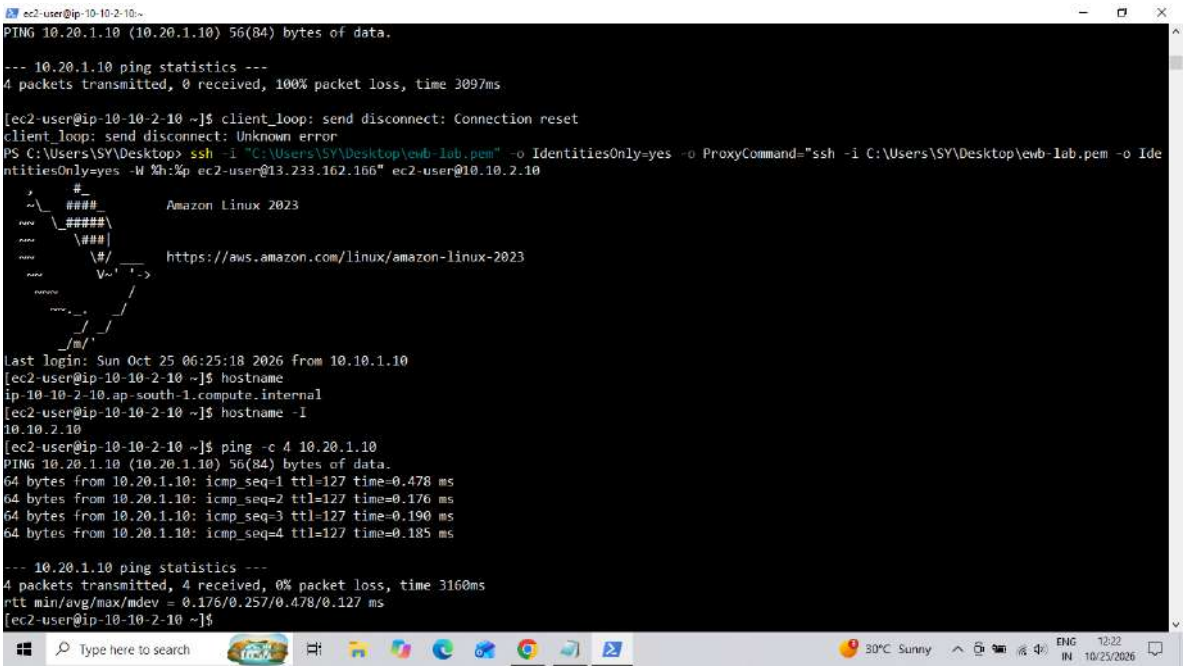
This command displays the IP addresses configured on the instance's network interfaces. In my test, the output included:

10.10.2.10

## Testing Connectivity Between App-A and App-B

From the terminal session on App-A, I ran:

ping -c 4 10.20.1.10



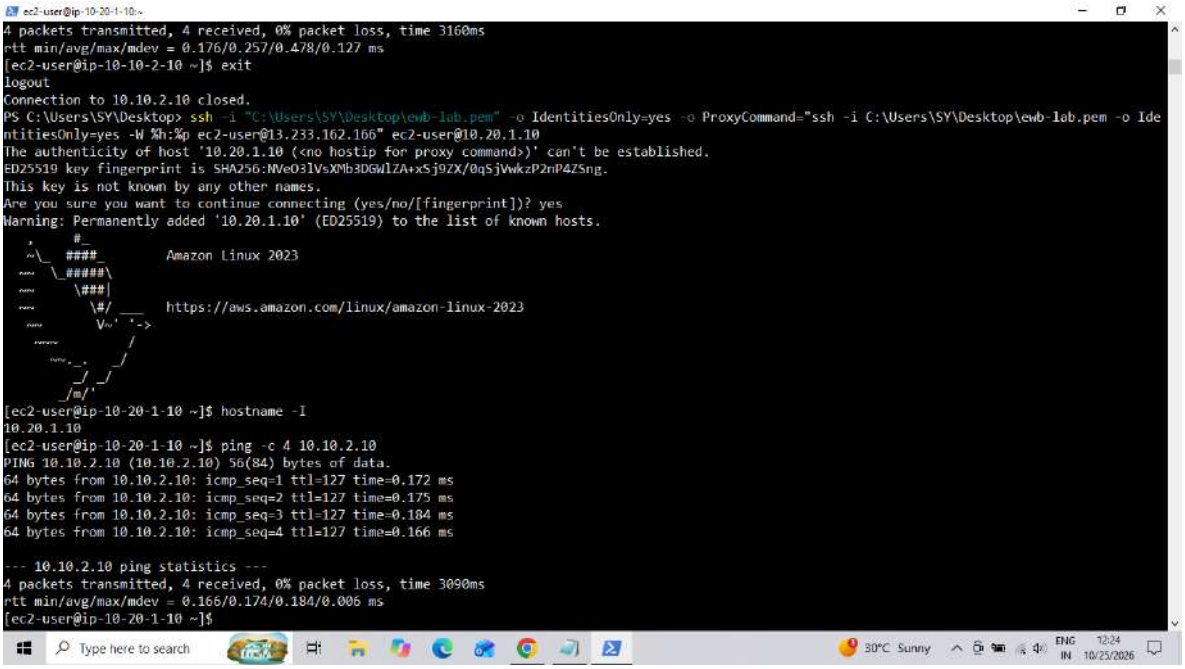

***Successful ICMP connectivity test from App-A (10.10.2.10) to App-B (10.20.1.10)***

# My test result

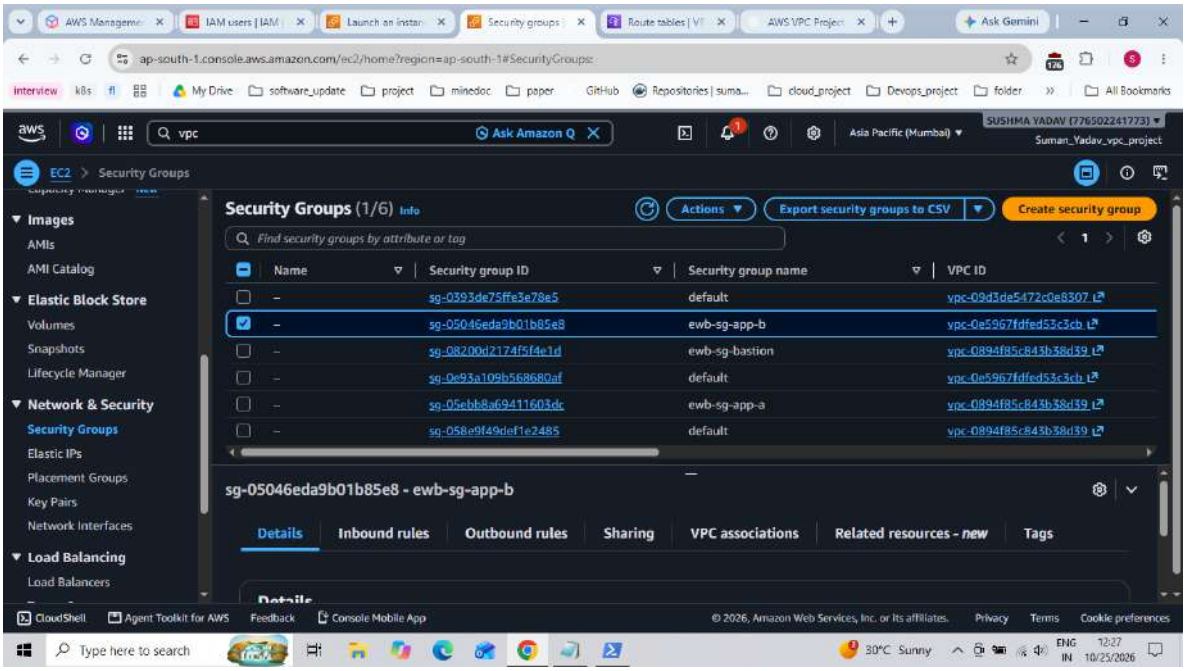
**I successfully pinged App-B from App-A.**

This confirmed that the ICMP connectivity test worked between the two private EC2 instances in separate VPCs.

# Connect to App-B

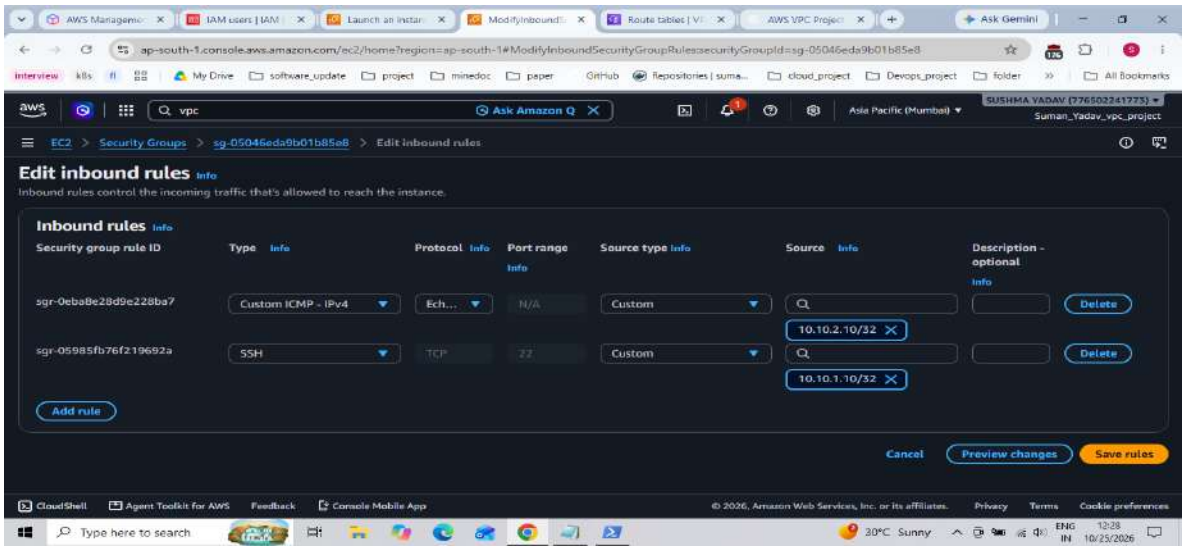



Successfully ping from app B too




## Controlled failure and recovery check ( NEGATIVE TEST)

Temporarily remove only the Custom ICMP - IPv4 Echo request rule whose source is 10.10.2.10/32.




## Return to App-A and run:

```
PS C:\Users\SY\Desktop> ssh -i "C:\Users\SY\Desktop\ewb-lab.pem" -o IdentitiesOnly=yes -o ProxyCommand="ssh -i C:\Users\SY\Desktop\ewb-lab.pem -o Ide
ntitiesOnly=yes -W %h:%p ec2-user@13.233.162.166" ec2-user@10.10.2.10
       #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sun Oct 25 06:50:13 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

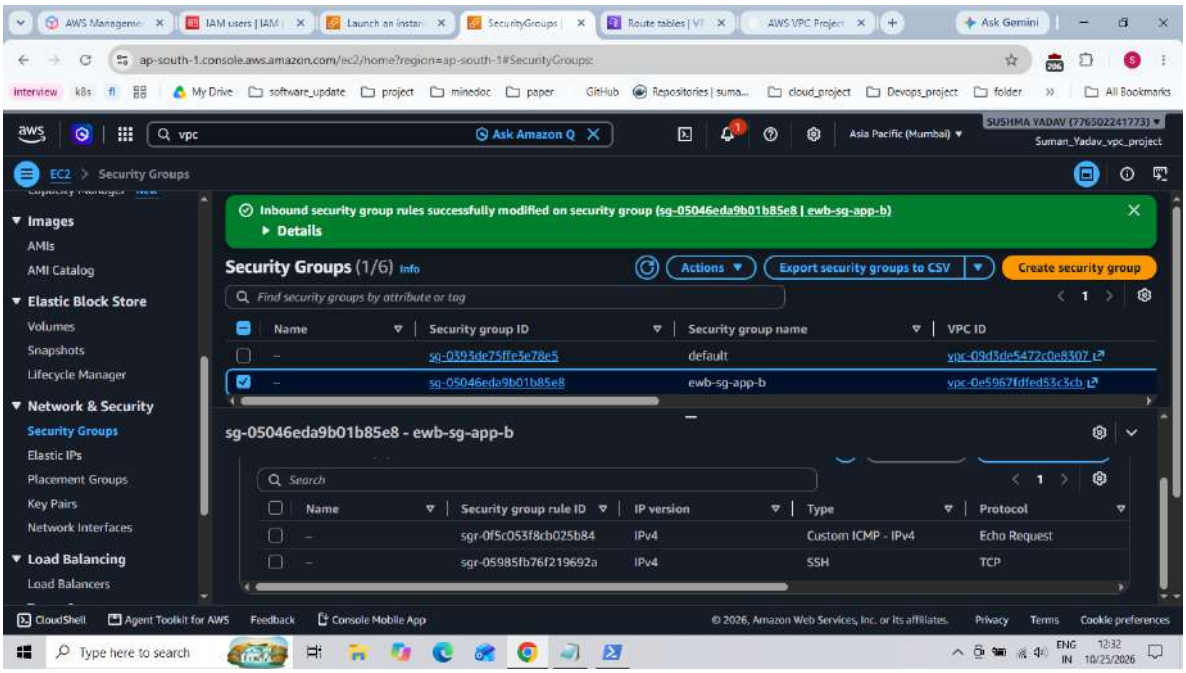
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3139ms

[ec2-user@ip-10-10-2-10 ~]$
```

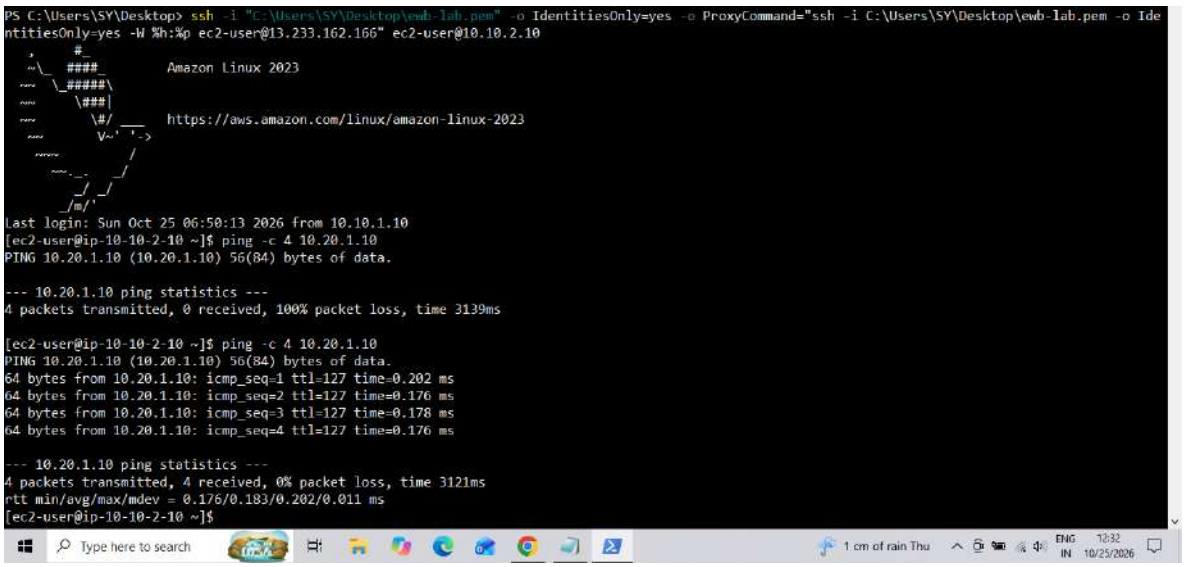
**Result: replies stopped because App-B no longer permits the ICMP Echo requests.**

## Now Restore the exact rule:

Custom ICMP - IPv4, Echo request  10.10.2.10/32




Now pinged successfully




Project Status: **Completed — VPC Peering connectivity was tested successfully between the application instances.**

## Conclusion:

Successfully implemented AWS VPC Peering and verified private communication between two VPCs using EC2 instances. This project strengthened my practical understanding of AWS networking, routing, security groups, and secure SSH access.